A FAST VIEWER FOR AOE SIMULATION RESULT FILES

visimu

Version 2.4.0

User and Reference Guide

The interface, the algorithms, the file format,
the C ABI and the scripting API

Paulo R. A. Gilz

Contents

About this guide **5**

1 What visimu is **6**
1.1 The shape of the program . . . 6
1.2 What the program does . . . 6

2 Getting started **8**
2.1 What you need . . . 8
2.2 Building and running . . . 8
2.2.1 Sample data . . . 8
2.2.2 Where the library is looked for . . . 9
2.3 Command-line options . . . 9
2.4 Logging, and what to do when it will not start . . . 10
2.5 The first plot . . . 10

3 The window **11**
3.1 Anatomy . . . 11
3.2 The main toolbar . . . 12
3.3 Mouse modes . . . 14
3.3.1 What a zoom drag zooms . . . 14
3.4 The status bar . . . 15
3.5 Files that change underneath you . . . 16
3.6 Theme . . . 17
3.7 Interface size . . . 17

4 The thirteen panels **19**
4.1 DATA — choosing runs . . . 19
4.1.1 Filtering the run list . . . 20
4.2 PARAMS — choosing the axes of a tile . . . 20
4.3 COMPUTE — channels the file does not contain . . . 21
4.4 DIFF — comparing two runs, and comparing two files . . . 22
4.5 ISO — filtering runs by a value they hold constant . . . 23
4.5.1 Grouping values into intervals . . . 23
4.5.2 Two constraints, composed . . . 24
4.5.3 What a workspace keeps, and what it does when the data has moved on 24
4.6 STYLE — colour, line, marker and legend . . . 25
4.6.1 Two encodings, chosen separately . . . 25
4.6.2 What decides a curve's colour . . . 26
4.6.3 The two tables . . . 27
4.6.4 Setting several rows together . . . 27
4.6.5 The legend . . . 27
4.6.6 Titles and axis captions . . . 28
4.7 SCALE — axis ranges and cross-tile constraints . . . 28
4.7.1 When two constraints meet on one axis . . . 29
4.8 SYNC — aligning the runs on an event . . . 30
4.9 3D — three channels against each other . . . 31

4.10 INSPECT — reading values off a curve . . . 31
4.11 AREA — shaded bands . . . 32
4.12 MARKERS — an instant, drawn two ways . . . 32
4.13 TEXT — captions on a tile . . . 32

5 Comparing two files **34**
5.1 Setting it up . . . 34
5.2 What must be true before anything is compared . . . 34
5.3 The configuration . . . 35
5.4 What counts as a violation . . . 36
5.4.1 Priority, and what gets shaded . . . 36
5.5 The report . . . 36
5.6 The pages it generates . . . 37
5.7 What the workspace keeps . . . 37
5.8 Trying it . . . 37

6 Pages, grids and tiles **39**
6.1 The page and tile bar . . . 39
6.2 How a page is modelled . . . 40
6.3 What a tile remembers . . . 40
6.4 Robustness when the files change . . . 42

7 Exporting, and saving your work **43**
7.1 Image . . . 43
7.2 CSV . . . 43
7.3 Workspaces . . . 43
7.3.1 New, Save, and Save as . . . 43
7.3.2 Names given to files and runs . . . 44
7.3.3 Loading against data that has moved on . . . 44
7.4 Background work, and why the interface freezes when it does . . . 45
7.4.1 Work that does not freeze anything: the sidecar builder . . . 45

8 The AOE file format **47**
8.1 Overall shape . . . 47
8.2 Two header dialects . . . 47
8.3 How runs are delimited . . . 48
8.4 Records, rows and wrapped lines . . . 48
8.5 Numeric encoding . . . 49
8.6 File extensions . . . 49
8.7 Files that are not AOE: HDF5 . . . 49

9 How the data is read **53**
9.1 Memory mapping . . . 53
9.2 One index pass, then lazy parallel columns . . . 53
9.3 The inner loop . . . 54
9.4 What happens to malformed input . . . 54
9.5 The `.aoec` sidecar . . . 54
9.5.1 Layout . . . 55
9.5.2 Deciding whether it is still valid . . . 55
9.5.3 Where sidecars live . . . 55
9.5.4 What it buys . . . 56
9.6 The memory budget . . . 56

10 Computed parameters: the formula language **57**
10.1 How an expression is executed . . . 57
10.2 Operators . . . 57
10.3 Element-wise functions . . . 58
10.3.1 The three logarithms . . . 58
10.4 Whole-series functions . . . 59
10.5 Constants and the lookup form . . . 59
10.6 Worked examples . . . 60
10.7 Names, dependencies and errors . . . 60

11 The algorithms behind the plots **62**
11.1 Decimation: drawing what the screen can show . . . 62
11.1.1 Selecting the visible window . . . 62
11.1.2 The pixel budget . . . 62
11.1.3 Statistics over the same window . . . 63
11.2 The redraw cache . . . 63
11.2.1 The resample key . . . 63
11.2.2 Adopting the drawn items . . . 63
11.2.3 The plot's own settings . . . 64
11.2.4 The drawn annotations . . . 64
11.3 ISO grouping . . . 64
11.3.1 Boundaries . . . 64
11.3.2 How values are written . . . 65
11.4 Legend order, and why it stopped shuffling . . . 65
11.5 Logarithmic axes . . . 66
11.6 The three-dimensional projection . . . 66

12 The C ABI **67**
12.1 Handles, versioning and errors . . . 67
12.2 Thread safety . . . 67
12.3 Function reference . . . 68
12.4 Building against it . . . 70

13 The Python scripting API **71**
13.1 The one rule . . . 71
13.2 Session . . . 71
13.3 Handles . . . 72
13.4 A worked script . . . 72

14 Performance, limits and troubleshooting **74**
14.1 Where the time goes . . . 74
14.2 Getting the best out of it . . . 74
14.3 Troubleshooting . . . 75
14.3.1 The program will not start . . . 75
14.3.2 A file opens but shows nothing . . . 75
14.3.3 The curves look wrong on a phase plane . . . 75
14.3.4 A value I expected is missing or reads as nan . . . 75
14.3.5 A channel disappeared after I removed a computed parameter . . . 75
14.3.6 Colours changed when I opened another file . . . 75
14.3.7 I reopened a workspace and the plot is empty . . . 76
14.3.8 I swapped one study for another and the axes moved . . . 76

A Keyboard shortcuts and gestures **77**
A.1 Keys 77
A.2 Gestures 77

B Environment variables **79**

C The workspace file **80**

D Defaults at a glance **82**

E Known limitations **83**

About this guide

This guide describes version 2.4.0 and nothing else. Every label, default, range and limit quoted here was read from the source of that version; where a genuine limitation exists it is recorded in a red box, and where a design decision is worth knowing about to use the program well, in a grey one.

It carries no figures: it is a text reference.

One version number is worth separating from the others before it causes confusion. **The application is at 2.4.0; the core library is at 2.0.0**, which is what `aoe_version()` returns and what `-version` prints as the core version. The C ABI has not changed since then — every change has been above the seam — so the core has not needed a new number.

Chapter 1

What visimu is

visimu reads AOE simulation result files — plain-text tables of many runs against many channels, routinely hundreds of megabytes — and plots them interactively. It is built as a C++17 core under a Python and Qt interface, and the seam between them is a plain C ABI, so the core is one shared library that any CPython 3.8 or later can load through `ctypes` and that any language with a C foreign-function interface can drive equally well.

1.1 The shape of the program

Layer	Responsibility
`core/` (C++17)	Memory-maps the result file, builds a sample index, parses columns on demand across a thread pool, evaluates computed parameters, decimates series for display, buckets ISO values into intervals, and reads and writes the `.aoec` sidecar. Owns all sample memory and the cache budget.
`core/include/aoe/aoe_c_api.h`	The seam: an opaque-handle C ABI with a thread-local error string. Everything above this line is written without a C++ compiler.
`python/visimu/_core.py`	Discovers and loads the shared library through `ctypes`, and wraps returned pointers in zero-copy NumPy views whose lifetime is pinned to the underlying block.
`python/visimu/session.py`	A Pythonic façade — `Session`, `FileHandle`, `SimulationHandle`, `IsoGroup` — which is also the public scripting API.
`python/visimu/model.py`	`AppState`: everything the core does not own. Pages, tiles, selections, ISO filter definitions, styles, annotations, scale constraints, and the workspace file format. Contains no Qt.
`python/visimu/ui/`	The Qt layer: the main window, the thirteen side panels, the plot grid and the plot tiles.

A rule visible throughout the source is that anything testable without a display lives in a Qt-free module. `model.py`, `scales.py`, `isogroups.py` and `projection.py` import no Qt, which is why the decimation budget, the log-axis conversion, the interval arithmetic and the orthographic projection can all be tested headlessly, and are.

1.2 What the program does

Six ideas account for most of the interface.

A page of tiles. The plotting area is a grid of up to twenty-five independent tiles, arranged from 1×1 to 5×5, and there may be any number of pages. Each tile is a complete plot definition: its own x channel, its own set of y channels, its own legend, its own axis locks, its own colouring and styling rules, and optionally its own three-dimensional projection.

Two independent constraints on what is drawn. The DATA tab chooses runs by name. The ISO tab chooses runs by the value some parameter holds constant across a run — a mass, a Mach number, a configuration. The two compose with a logical AND: a curve is drawn if and only if its run is ticked in DATA and admitted by every active ISO filter. Neither tab ever writes into the other's state.

Two independent encodings of what is drawn. Colour names one identity and the dash pattern names another, chosen separately per tile. Several files of the same parameters on one tile is the case this exists for: colour by parameter, style by file, and the eye reads both at once.

Channels that need not exist in the file. A computed parameter is an expression over the channels that are there — `deriv(THETA, TIME)`, `NZ / max(NZ)`, `ALTITUDE[TIME=last] - ALTITUDE[TIME=first]` — evaluated in the core, per run, and thereafter indistinguishable from a channel the file carried. The same mechanism gives a cryptic column a readable name — `Load factor = NZ` — without touching the data.

The difference between two runs, where the runs themselves are too alike to tell apart. A tile can draw one channel's difference between two runs, which may come from different files and need not have been recorded at the same instants: the second is read at the first's instants by interpolation, over the stretch they have in common. See Section 4.4.

Two whole files, checked against written-down thresholds. The same question asked of everything at once: every run of one file against the run of the same name in another, every channel somebody wrote a threshold for, and a report of what moved further than it was allowed to. The thresholds live in a file of your own, the report lists one line per run, and the tiles it generates on request overlay both files with the crossings shaded. See Chapter 5.

Drawing what the screen can show, exporting what the data says. A tile with twenty-five runs of five hundred thousand samples does not hand two and a half million points to Qt. It asks the core for a min/max decimation of exactly the visible window at roughly two points per horizontal pixel, so that narrow spikes survive but the item count stays in the low thousands. The CSV export does the opposite and writes every sample, because a plot is a picture and an export is data.

Chapter 2

Getting started

2.1 What you need

Building the core requires nothing but a C++ compiler — GCC 8 or later, Clang 7 or later, MSVC 2019, or MinGW-w64 — and GNU make (or `mingw32-make.exe` on Windows). CMake is supported as an alternative but is not required. Running the graphical application additionally needs three Python packages: `numpy`, for array views over the core's buffers; `PySide6-Essentials` together with `shiboken6`, for the Qt widgets; and `pyqtgraph`, for the plotting canvas. Building the standalone executable adds `PyInstaller`.

2.2 Building and running

```
# Build the core and the single-file executable
./build.sh                  # Linux and macOS
build.bat                   # Windows (MinGW)

# Check that it works
./build.sh test             # or: build.bat test

# Run from source, without packaging
PYTHONPATH=python python -m visimu                # Linux and macOS
set PYTHONPATH=python && python -m visimu         # Windows
```

With no argument both scripts do the whole job: they compile the core and then package the executable, because that is what "build" means and it used to be two commands. The pieces are still available on their own — `core` compiles the library and stops, `exe` packages without recompiling, `exe-folder` packages as a directory rather than one file, and `test` compiles the core and runs the C++ checks. Anything beginning with a dash is passed straight through to `packaging/build_exe.py`, so `./build.sh --no-samples` works as written.

The build produces `build/libaoecore.so` on Linux, `build/libaoecore.dylib` on macOS and `build/aoecore.dll` on Windows. Two directories are generated rather than kept in the repository: `build/`, holding the compiled core and the PyInstaller work files, and `dist/`, holding the frozen executable.

To try the program without real data, generate a sample:

```
python tools/gen_sample.py sample.aoe --sims 6 --rows 40000
PYTHONPATH=python python -m visimu sample.aoe
```

2.2.1 Sample data

Building the executable also writes `sample/`, beside `build/` and `dist/`. Two kinds of file go in there, for two different purposes.

Flight data, to look at: two files of about 250 MB with twenty-five runs each, and one of 12 MB for a quick look.

A regression pair, to compare: `regression_a.aoe` and `regression_b.aoe`, about a megabyte each, twenty channels and four runs held under the same identifiers in a different order in each

file — plus `regression_thresholds.yaml`, the configuration that goes with them. They are built wrong on purpose and in known places: one run is identical in both files and must come out clean, one is flagged on eighteen channels, which is more than one page of tiles holds, one on five and one on a single channel. Chapter 5 is what they are for, and they are generated by `tools/gen_diff_samples.py`.

The sizes are the point. A few megabytes exercises none of what this program is built around: the `.aoec` sidecar is not written below 8 MB, decimation has nothing to decimate, and a six-run file has no sweep for ISO to filter. Two large files rather than one, so that *Colour by Parameter* with *Style by File* has something to show.

Only the files that are missing are generated, and one already there is never rewritten — these take minutes to integrate, and someone who has trimmed one down or replaced it with their own data should not find it overwritten by a rebuild. Each is written under a temporary name and moved into place, so an interrupted run leaves nothing that looks finished; without that, a Ctrl+C halfway through would count as "already there" on the next build and the viewer would open a truncated file.

Failure here never fails the build: a sample file is a convenience, not part of the product, so a read-only checkout or a full disk prints a reason and the executable is produced anyway. `--no-samples` skips the step.

The generator is `tools/gen_flight_data.py`, which integrates a longitudinal rigid-body model with a pitch-rate autopilot rather than emitting noise — test data made of sine waves lets a viewer look correct while being wrong in the ways that matter, since it has no transients that decimation must preserve and no per-run constants for ISO to key on.

2.2.2 Where the library is looked for

`_core.library_path()` searches in a fixed order, most specific first: the environment variable `VISIMU_CORE` (which may name either a file or a directory); `sys._MEIPASS` and `sys._MEIPASS/visimu`, which is where a PyInstaller bundle unpacks; the directory of the executable when frozen; the package directory; `<project>/build`; and finally the working directory. If none of them yields a loadable library, the raised `CoreError` lists every directory that was searched and suggests building the core first. On Windows a load failure that looks like a missing dependency adds a hint to run `objdump -p "<path>" | findstr "DLL Name"`.

2.3 Command-line options

Four options are recognised, and two of them are diagnostic tools worth knowing before you need them.

Option	Effect
`files...`	Result files to open at startup, given as positional arguments.
`-version`	Prints the application version (2.4.0), the core version (2.0.0), the path of the core library actually loaded, the worker count and the log path.
`-selftest`	Imports `numpy`, `PySide6`, `PySide6.QtWidgets` and `pyqtgraph` one at a time, loads the core through `ctypes`, then constructs a `QApplication`, falling back to the offscreen platform when there is no display. Prints `OK` or `FAIL` per component, then either *All checks passed.* or a count of failures. This is the first thing to run when the program will not start.
`-memory-limit MB`	Cap on cached sample data, default `2048.0`, floored at 64 MB. Columns beyond the cap are dropped least-recently-used first and re-read on demand.

2.4 Logging, and what to do when it will not start

The logger is named `visimu` and writes `visimu.log` through a rotating handler of 2 MB with three backups. The directory is chosen in order: the environment variable `VISIMU_LOG_DIR`; the directory of the executable when frozen, or the project root when running from source; `~/.visimu`; and the working directory. On startup the log records the application and core versions, the core library path, the Python and platform versions, the worker count and the PySide6, pyqtgraph and NumPy versions — which between them identify almost every environment problem.

The application installs `sys.excepthook`, `threading.excepthook` and a Qt message handler, so an unhandled exception anywhere, including inside a Qt callback, reaches the log rather than the void. If construction of the main window fails, the traceback is logged and printed, and a dialog titled *visimu could not start* appears with the log path as its informative text and the traceback as its detail.

2.5 The first plot

When the program opens with no arguments the window is empty: a grid of one tile labelled *empty tile*, and a DATA tab with nothing in it.

Getting to a plot takes two steps. Press *Add files* in the DATA tab and choose one or more result files; each appears as a top-level row with its runs beneath it, and every run is ticked by default. Then open the PARAMS tab, where the x axis has already defaulted to the first channel whose name begins with `TIME`, and tick one or more y channels. The tile draws as soon as it has both an x channel and at least one y channel.

Opening a file of any size is done on a worker thread. For files of 8 MB or more, once the file is open and on screen a *separate process* is started to write an `.aoec` sidecar beside the result file; the status bar says so and the window stays usable throughout. That happens once; subsequent opens of the same file skip the index scan entirely and are several times faster. Chapter 9 describes the mechanism.

Chapter 3

The window

3.1 Anatomy

The window has six regions. Across the top is the main toolbar, holding workspace load, save and save-as, the two exports, the auto-fit button and — pushed to the right edge — the four mouse modes. The title bar above it names the workspace currently open, with a leading asterisk while there are unsaved changes. Down the left edge runs the tab rail, a 21-pixel strip of vertically painted labels which selects which of the thirteen panels is shown beside it. It holds tabs and nothing else — the light/dark toggle is in the toolbar, where it does not compete with the tabs for the one dimension a short screen is short of.

There are four ways to reach a panel, because thirteen names down a 21-pixel column is a lot of aiming. Click a name; press its letter; roll the wheel over the names; or walk them with the arrow keys once the rail has the focus, which clicking a name gives it. Home and End go to the first and last. Dragging a name up or down rearranges the rail, and the order is saved with the workspace. The

D	DATA	O	ISO	I	INSPECT
P	PARAMS	S	STYLE	R	AREA
C	COMPUTE	A	SCALE	M	MARKERS
F	DIFF	Y	SYNC	T	TEXT
		3	3D		

The letters are taken from the labels wherever the obvious one was free: ISO gives up I to INSPECT, which is reached far more often, and takes its O; SCALE takes its A, since AREA has R. Each name's tooltip says its letter, so none of this has to be remembered.

A bare letter, and how it keeps out of the way of typing

A letter with no modifier, because these are navigation rather than commands: thirteen more Ctrl bindings would collide with the window's own.
That makes the collision with *typing* the thing to get right, and a `QShortcut` is the wrong tool for it — a shortcut on a plain letter wins against the widget holding the focus, so typing a page name would jump tabs instead of writing a "d". A handler on the window is no better: a list or a table takes letters for its own type-ahead search and never passes them up.
So the key is read by a filter on the application, before anyone else sees it, and handed straight back whenever something is being typed into — a line edit, a spin box, an editable combo, a text area, or a cell editor, which is a line edit parented to a table. Anything custom is asked the general question Qt answers for itself: whether the widget has input-method handling enabled. A modifier held, or a modal dialog up, hands the key back too.

Why the rail may be rearranged, and what does not move with it

Which panel a name opens never changes. Order is by position and identity is by panel index, and the two are kept apart deliberately: everything outside the rail — the panel

stack, the window's tab handler, the workspace — refers to a panel by the index it was built with, so dragging the rail about cannot make the window show the wrong panel. A workspace records the order as a list of *names* rather than positions. A name this build does not have is ignored, and one it has that the workspace does not mention keeps its place at the end — so the rail is arranged as far as the file says and no tab is ever lost from it.

The panel itself occupies a splitter pane that can be dragged between 230 and 600 pixels wide. To its right, the plot area is divided into the page and tile bar along its top and the grid of tiles below. The status bar along the bottom carries, from left to right, the description of any running background task or the standing summary of what is loaded, a progress bar while a task runs, the crosshair readout, a note of any sidecar being built in another process, and the size of the sample cache.

The active tile is the one every per-tile control acts on, and it is marked by a two-pixel accent border rather than a one-pixel neutral one. Clicking any tile makes it active. That border is set through a guard that returns early when the state has not changed, which is not a micro-optimisation: `QWidget.setStyleSheet` performs no comparison of its own and repolishes the widget and all its children, and since a tile is a `QGraphicsView` that means a full scene repaint. Without the guard, one click on a 5×5 page would repaint twenty-five plots.

3.2 The main toolbar

Fifteen buttons, left to right in five groups. The workspace comes first and carries no caption, because a new, a load and two saves need none. Then *Export*, the two ways work leaves the program. Then *Mouse*, the four modes. Then, after a stretch that pushes the rest to the right edge, *Help* on its own, and *Interface* last: the theme and the two size steps. Every button is a labelled icon, and every icon is painted at request time rather than loaded from a file.

Button	Shortcut	What it does
New workspace	Ctrl+N	Starts again from nothing: no files, no pages but one, no filters. Offers to save the open workspace first.
Load workspace	Ctrl+L	Opens a saved `.aoews` workspace, replacing everything currently loaded. Offers to save the open one first.
Save workspace	Ctrl+S	Overwrites the current workspace, asking for a path only if there is not one yet. The tooltip names the file it would write.
Save workspace as…	Ctrl+Shift+S	Always asks for a path, then continues in the new file and leaves the old one as it was.
Image	—	Saves the active page as a PNG, exactly as displayed.
CSV	—	Writes the active tile's curves to a CSV file at full resolution.
Help	—	Opens the shortcut and panel reference: every key the window listens for, and a line on what each tab is for.
Light / Dark	—	Switches the theme. Its label and icon name the theme a press would *give* — a sun while the dark theme is on — rather than the one already on screen.
Smaller	Ctrl+-	Interface size: smaller text, tighter spacing, everywhere.
Larger	Ctrl+=	Interface size: larger text, roomier spacing, everywhere.
Select	Ctrl+1	Mouse mode: click to activate a tile; drag annotations.
Inspect	Ctrl+2	Mouse mode: click a curve to read every channel; drag to measure a slope.
Pan	Ctrl+3	Mouse mode: drag to move the view.
Zoom	Ctrl+4	Mouse mode: the wheel zooms; a drag zooms the area, the x axis or the y axis according to its shape.

There is deliberately no *Open* action: *Add files* in the DATA tab and *Load workspace* here already cover it.

The interface group is last and hard against the right edge, behind its own rule and under its own caption, because the theme and the two size steps are the only buttons here that change nothing about the data or the view — everything to their left acts on the workspace, the exports or the plots, and these three act on the window itself. *Help* sits just before them, separated from both: it is not a view setting and not something that acts on the work.

New, *Load* and closing the window all go through the same question. If the open workspace has unsaved changes they offer to save it first — *Save*, *Discard* or *Cancel* — and choosing *Save* on a workspace that has never been written opens the file dialog. Cancelling that dialog cancels the whole action rather than continuing and losing the work, which is the one case where "save" quietly turning into "discard" would be unforgivable.

The toolbar is permanent by construction. Its context menu is suppressed, its `toggleViewAction` is disabled and hidden, and `MainWindow.createPopupMenu` returns nothing, because hiding the only toolbar in a window that has no menu bar would leave no way to bring it back. Its icons are not image files: they are painted on demand by `ui/icons.py` onto a 100×100 canvas and cached by name, colour and size, so they follow the theme and survive a PyInstaller bundle that ships no assets.

3.3 Mouse modes

The four modes are exclusive, and the inactive ones are genuinely off rather than merely ignored — in Select and Inspect the view box has mouse interaction disabled on both axes, and the wheel event is swallowed. Switching mode writes a one-line hint into the status bar.

Two gestures work in every mode, because neither competes with what the mode already means. **Double-click** a tile to make it active, fit it and bring PARAMS forward — three things from the gesture that means "this tile, now". **Double right-click** anywhere on the grid to fit every tile on the page, which is Ctrl+0 without reaching for the keyboard.

Both are armed on the second press and carried out on the release, and only if the pointer stayed put. Qt reports the second press of two quick clicks as a double click whether a drag follows it or not, so in Zoom mode — where dragging *is* the gesture — a second rectangle dragged near the first would otherwise arrive here and the fit would throw the zoom away. A double click that turns out to begin a drag now does nothing at all.

Switching into Inspect brings the INSPECT panel to the front, and clicking a curve brings it forward again every time, because that is where the answer appears. Leaving Inspect clears the inspection dot. A locked axis stays locked whatever the mode says: the limits are re-applied and per-axis mouse interaction disabled on every redraw, so no drag or wheel gesture can escape a range you have frozen.

Mode	Drag does	Wheel	Annotations
Select	Moves markers and the edges of shaded areas; moves the legend and text captions.	inert	movable
Inspect	Rubber-bands a slope line; a drag shorter than six pixels counts as a click and inspects instead.	inert	frozen
Pan	Pans the view. On a 3D tile: turns it; middle-drag or Shift +drag slides it; right-drag or Ctrl +drag zooms.	inert	frozen
Zoom	Rubber-bands a region to zoom into — the area, the x axis alone or the y axis alone, according to the shape of the drag. With Ctrl , or with the right button, every tile follows on x.	zooms	frozen

Annotations are movable only in Select mode and only on a two-dimensional tile.

3.3.1 What a zoom drag zooms

In Zoom mode the shape of the drag decides which axes it applies to.

Drag	Result
An area	Zooms into that rectangle.
A flat line across	Zooms the **x axis only**; y is left exactly as it was.
A line up and down	Zooms the **y axis only**; x is left exactly as it was.

A drag counts as a line when its minor dimension is at most 18 pixels at a scale of 1.0 *and* at least four times shorter than its major one. Both tests are needed: the ceiling alone would call a 20×16 drag horizontal, and the ratio alone would call a 400×60 one horizontal when whoever drew it plainly meant a region. The ceiling follows the interface scale, so a line means the same thing to the eye at 70 % as at 160 %. A drag smaller than six pixels in both directions does nothing at all, so a twitch while clicking cannot zoom the view by a factor of a hundred.

The rubber band shows the decision as it is made — spanning the full height of the tile for an x-only zoom and the full width for a y-only one — so the gesture is confirmed before the button is released rather than after. There is no mode to select and nothing to hold down.

Why the constrained gestures exist

The unconstrained one is usually not what is wanted. A stretch of x with y untouched is the common case — a manoeuvre between two instants, every channel of it still in frame — and doing that with a rectangle means dragging the full height of the tile by eye and getting it slightly wrong, which rescales y by a few per cent for no reason and makes two tiles that should be comparable no longer comparable.
Internally all three are the same operation. An x-only drag is widened to the full height of the view before it is mapped into data coordinates, and such a rectangle gives back exactly the y range the axis already had — so there is one code path rather than three, and it is the one that was already correct.

Dragging on an axis itself still means "this axis only", and is handled by pyqtgraph as before.

Zooming every tile at once

Hold Ctrl, or drag with the *right* button, and the x range the zoom settles on is given to every other tile as well. Whatever the shape of the drag: an area zoom hands over the x it took and leaves each tile's own y alone, and an x-only drag does the same. The gesture asks "show me this stretch in all of them" — which is the usual question of a page of tiles, since they are nearly always plotted against the same x.

Three kinds of tile stay where they are, each for a reason the user would give themselves: a 3D tile, whose frame is a box rather than a pair of ranges; a tile whose x axis is locked in SCALE, because a lock is an instruction; and a tile plotted against a *different* x channel, where the numbers of one axis mean nothing on the other. The status bar says how many tiles followed.

Why it is read off the event rather than set as a mode

There is nothing to switch on beforehand and nothing left switched on afterwards, which matters for a gesture whose effect reaches twenty-four plots the user is not looking at.
The range is applied with each tile's range events suppressed, and the window announces the change once when the round is done. Twenty-five tiles each announcing one would have the SCALE boxes rewritten and the INSPECT statistics — which read every selected channel of the visible window — recomputed twenty-five times over a single drag.
A right *click* that was never a drag is left alone, so the view box's own context menu still opens.

3.4 The status bar

The standing summary reads *N file(s), P run(s) plotted, M parameter(s)*. When an ISO filter is holding runs back it becomes more explicit — *4 file(s), 17 of 25 run(s) plotted, 8 held back by the ISO filter(s), 42 parameter(s)* — which is the shortest honest description of a state that would otherwise look like lost data.

The progress bar is 120 pixels wide and takes one of two forms, according to whether there is anything honest to count.

A task with a known number of steps — opening seven files, re-reading four, loading a workspace that names three — gets a determinate bar reading *3 of 7*, which advances as each file is finished. A task with no step count gets range 0 to 0, Qt's indeterminate form, with no text at all: the core reports no progress from inside a single parse, and a fabricated percentage would be worse than an honest *working*.

Freezing the interface during a task is deliberate: an action taken while a task runs would be lost, so it is better refused than accepted and dropped. What matters is that the difference between *busy* and *dead* is visible, which on a five-minute task is the difference between waiting and reaching for the task manager.

So the pointer becomes a wait cursor for as long as the interface is disabled — wherever it happens to be, which the status bar cannot reach — and the bar counts steps whenever steps are countable. The cursor is tied to the disabled state rather than to the task, because a chain of queued tasks starts several times and finishes once, and pushing a cursor per task would leave the hourglass on for good.

Further right, *caching N files in the background* appears while a sidecar is being written in another process. It is deliberately not the progress bar: the bar means *the window is waiting*, and the whole point of building a sidecar outside the application is that the window is not. Its tooltip says as much. On the right, *cache NN MB* is the sample data the core is holding; its tooltip explains that columns are dropped automatically when the figure exceeds the limit and re-read when needed again.

3.5 Files that change underneath you

Nothing polls the disk. Whether the data on disk is finished is a question only the person running the solver can answer, so they answer it: a file is re-read when you press *Reload*, and at no other time.

A solver halfway through rewriting its output is a file that has changed and must *not* be read, so a prompt offering to reload it would be asking a question it has no business asking — and asking it while you are reading a plot of something else, where answering is the only way back.

Reload, in the DATA tab, re-reads the selected file or files from disk; with nothing selected it re-reads every open file, which is what "the solver has just finished" usually calls for. Selecting a run means the file it belongs to — there is no such thing as reloading one run. The same action is on the row's right-click menu, named after the file it would re-read.

A reload drops every cached sample for those files and re-reads the index, so it costs a full parse and runs on the worker thread exactly as opening does; a sidecar the new contents have invalidated is rewritten in the same pass, which spends the parse the user is already waiting for rather than the next open's.

Names survive it. The file keeps the name you gave it, every run keeps its own, and so do their colours and dashes.

Why a reload keeps the styles of a renamed file

A run's key is `<file>::<id>::<ordinal>`, and `reload()` re-opens the dataset and rebuilds every key. If the `<file>` part were the *display* name, renaming a file and then reloading it would rename every one of its run keys with it — and every colour, dash and marker chosen for them would suddenly be attached to a key nothing answered to. The key is built from the name the file was *first* given, which renaming does not touch and a reload does not change.

On closing, if anything has been modified, a three-way prompt titled *Unsaved changes* offers to save the workspace first; *Cancel* aborts the close. The application then waits up to thirty seconds for background work to finish before destroying the session, because tearing the core down while a worker is still inside it would write into freed memory.

3.6 Theme

The theme button is in the toolbar's interface group (§3.7), and flips the whole application: the plot canvases, the legend grounds, the painted toolbar icons and the text captions.

The toggle's label and icon name the theme a press would give rather than the one already on screen — a sun while the dark theme is on — because a toggle labelled with its current state reads as a description, and a button is an offer.

The stylesheet sets every colour it depends on instead of inheriting from the desktop, because Qt's default palette differs enough between Linux and Windows that an application which is correct on one can be unreadable on the other. Two details are worth knowing. Check and radio indicators are drawn explicitly, including the ones inside tree, list and table views — those are drawn by the views rather than by `QCheckBox`, and they are the ticks beside a run in DATA, a parameter in PARAMS and a value in ISO. And the type scale exists only as a set: widget text is sized in pixels (10 px body, 9 px for hints, section titles and the status bar) while plot text is sized in points (7.5 pt tile titles, 7 pt axes, 6 pt legend rows, 6.5 pt readouts and captions), because a stylesheet that sets a pixel size makes `QFont.pointSizeF()` report -1 and any code that reads a font back would have to guess.

Legend rows are 6 pt, a point below the axes, with little padding inside the legend's frame and little space between a row's swatch and its text. A legend is a key that is read once and glanced at thereafter, and at the axis size it took a visible corner out of every tile on a 3×3 page.

3.7 Interface size

Smaller and *Larger* in the toolbar scale the whole interface, from 70 % to 160 % in steps of ten points. A session starts at 90 %: the base figures were chosen against a 1080-high screen and are a little generous on most of the screens this is used on, and one step down fits the whole interface on every screen it has been run on. The setting is saved with the workspace, like the theme, and a workspace that names a size opens at the size it was saved at.

One knob rather than a set of them. Text set two pixels smaller inside unchanged padding looks shrunken rather than roomier, so the padding, the tick boxes, the scrollbars and the rail move with the type or none of it should. It reaches:

- the application stylesheet — every font size and every padding figure in §3.6 is a base size multiplied by the scale;
- the widgets that paint their own text, which measure it themselves and so are not reached by a stylesheet. The tab rail is the one that shows: its *width* is the height of its rotated labels, so it re-measures and the column grows or shrinks with them;
- the plots — tile titles, axis tick numbers, legend rows, the crosshair readout and the text captions are all sized in points handed to pyqtgraph, and the legend's own padding moves with them.

What it does not touch is anything measured in the data: curve widths, marker sizes and band opacities are choices about the plot rather than about the window, and they have their own controls in STYLE, MARKERS and AREA.

Each button disables itself at its end of the range. A button that silently does nothing is worse than one that says it cannot: the scale clamps, so without that the last press at either end reads as the window failing to respond.

Why this exists

The base sizes are one compromise, chosen for a 1080-high screen, and screens are not all that. This was reported from a 1400 × 1050 Linux session, where the window overflowed the screen and could not be maximised; a 4K screen at 100 % has the opposite problem. Scrollable panels (below) make the window fit on any screen; this makes it *comfortable* on one.

Chapter 4

The thirteen panels

The tab rail selects one of thirteen panels. Selecting a tab refreshes that panel immediately and marks the others stale, so they are refreshed lazily when next shown — which is why no panel is allowed to write application state while refreshing. Six act on the active tile only (PARAMS, STYLE, SCALE, SYNC, 3D, TEXT) and the rest are global. DIFF is both: its difference half acts on the active tile, and its file comparison is about the files as a whole.

Each panel sits in a scroll area, and the reason is the window's minimum size rather than the panel's own.

A plain widget's minimum height is the sum of its children's, and STYLE is tall: four switches, two encodings with their combos and hints, two tables and a legend group. Added to the toolbar, the page bar and the status bar, that would put the window's minimum height near 950 px — fine on a 1080-high screen, and taller than the usable area of a 1050-high one once a desktop panel takes its share. The window could then be neither shrunk nor maximised, because a maximise that cannot satisfy the minimum is refused outright.

A scroll area does not propagate its child's minimum height, so the window's minimum does not depend on the tallest panel. Nothing is hidden: a panel keeps its full height and scrolls only when the pane is shorter than it. Horizontal scrolling is off, so a control can never be parked off to the side where it cannot be found.

4.1 DATA — choosing runs

The panel is a tree with one top-level row per open file and one child row per run. *Add files* opens the file dialog, which filters on AOE result files (`*.aoe *.dat *.txt`). *Remove* unloads the selected files, and acts only on file rows — selecting a single run and pressing *Remove* must not unload the whole file it belongs to. *Clear all* unloads everything.

Ticking a run plots it. Ticking a file row plots every run in that file, and clearing it plots none of them; the box is tri-state, so a file with only some of its runs ticked shows the indeterminate mark. The same two operations are on the context menu by name — *Plot all 25 runs in flight_b.aoe* and *Plot no runs in flight_b.aoe* — along with *Rename…*, *Reload … from disk* and, on a file row, *Copy full path*. At the foot of the panel, *Plot all* and *Plot none* act across every loaded file, driven from the session rather than from the tree. *Reload*, beside *Rename*, re-reads files from disk; §3.5 describes what it does and why nothing does it on its own.

A file of more than 200 runs arrives with its runs hidden until you open it. Expanding a node is what makes Qt lay out and build view items for its children, so a twelve-hundred-run study — the size the performance chapter measures — would otherwise pay for all twelve hundred rows on load, before anyone had looked at one. Nobody scrolls through twelve hundred rows to find a run either.

Nothing is missing: every run is in the tree, its tick is in force, and *Plot all* still reaches it. Whichever way you leave a file, open or closed, is remembered — by path, so it survives a reload and a rename.

Renaming, by double-click or from the menu, changes the display name only. The identity a style attaches to is a separate key that never changes, so recolouring does not follow a rename; the tooltip of a renamed run records what the file calls it.

A name given here is written into the workspace and restored when it is loaded. §7.3 describes how the names are keyed.

Why the list does not blink

The tree is rebuilt only when its structure changes — a signature over each file's index, name, path and the index, name and file-level identifier of each of its runs. Ticking a box leaves every widget in place and writes only the check states, so scroll position and expansion survive. Before this, selecting a run rebuilt the whole tree, which read as a flicker and lost your place in a twenty-five-run file.

A run that is ticked here but excluded by an ISO filter is painted grey, and its tooltip gains a sentence explaining that it is held back rather than unselected. That greying is the visible half of the two-constraint model described in §4.5.

4.1.1 Filtering the run list

Runs are often named `X.Y.Z` and often number in the hundreds, so the slow part of this panel is scrolling. Typing in the box hides what does not match.

A bare pattern matches any part of the name: `2.1` shows `2.1.3` and `1.2.1` alike, because the middle field is as often the interesting one as the first. A pattern containing a wildcard is matched against the whole name instead, so `2.*.3` means first field 2, last field 3. Matching ignores case, and a file whose own name matches keeps all of its runs — having asked for that file, hiding most of it would be perverse.

Every filter box in the program — here, in PARAMS and in INSPECT — has a button beside it that empties it and refilters at once. Qt's own clear icon appears inside the field as well, but only once the field has text in it, which is no use to someone looking for the way back before they have typed.

It is a view and nothing else: a hidden run keeps its tick and goes on being plotted, and clearing the box brings it back exactly as it was. While a filter is typed, *Plot all* and *Plot none* read *(shown)* and act only on the rows on screen; "show me 2.1*, then plot all of them" is the reason to have a filter at all.

4.2 PARAMS — choosing the axes of a tile

PARAMS sets the x channel and the y channels of the active tile, and nothing else: the legend, grid, log scale and marker settings live in STYLE. Every control here displays the channel's label — the name with its unit appended, as in `ALTITUDE [ft]`, when the file's dialect carries one — and carries the bare name as its data, so a unit can never be mistaken for part of a name.

The x channel is a combo at the top. Under *Y axis* are two lists, stacked: *Available*, holding every channel the open files offer, and *Plotted on this tile* beneath it, holding the tile's y channels. Three buttons sit between them:

Button	What it does
▽	Plots every name selected in *Available*.
△	Stops plotting every name selected in *Plotted*.
×	Plots nothing on this tile.

The lists are stacked rather than side by side, which is what makes the two arrows read correctly: down means "plot these", up means "take them off". Double-clicking a name moves it the same way, and is the quickest route for one channel. Both lists take a multiple selection, with Ctrl and Shift as everywhere else, so a dozen names move in one gesture.

Two lists rather than one list of tick boxes. A tick box in a six-hundred-row list is a 13 px target that has to be hit one row at a time, and the question "what is on this tile" was answered by scrolling the whole list looking for ticks. Here it is answered by looking at the second list.

Available stays in the file's own order — the order the engineer reading that file expects — whatever order names were taken off the tile in. *Plotted* is in drawing order, which is the order names were added to it, and that order is the tile's y list: it decides what the title names first, the order of the legend rows and the column order of the CSV export. To move a channel later in that order, take it off and add it again.

Each list has its own filter box, each with its own clear button. A filter matches against the channel name, uppercased; if the text contains `*` or `?` it is a wildcard pattern, otherwise a substring test. Filtering hides rows and changes nothing else — a plotted channel the filter is not showing is still plotted, and an available one still available.

Each refilters 400 ms after the last keystroke rather than on every one. These files carry hundreds of channels — 585 in one of the dialects this reads — so a letter is several hundred rows hidden or shown and a relayout of the list, and the answer between two letters of the same word is not one anybody reads. It is the same delay, for the same reason, that the TEXT panel uses before it commits a caption.

Right-clicking a channel in either list offers *Use as X axis* and *Copy name*; COMPUTE is where a channel is given a readable name.

Displayed default versus committed default

Refreshing the panel displays a sensible x channel — the first whose name starts with `TIME`, otherwise the first channel — but does not write it to the tile. The default is committed once per redraw, by `AppState.apply_parameter_defaults()`, and only for tiles that already have a y or z channel. Blank tiles are left blank.

4.3 COMPUTE — channels the file does not contain

A computed parameter is a name and an expression. Type both and press *Create*; the expression is validated on every keystroke, and the line beneath the field reads *Valid* or carries the core's parse error. Validation evaluates nothing — it parses and resolves identifiers only — so it is cheap enough to run while you type.

The expression field completes channel names as you type. From the second letter of a word it offers the channels whose names begin with it, ignoring case — the file's own channels and the computed ones alike — and choosing one replaces *the word under the cursor* rather than the whole field, so completing in the middle of a long formula works. One letter is not enough to offer on: it matches most of a 585-channel file, which is a list to scroll rather than a suggestion.

It offers only names. Function names are in the reference behind *Help*, and a list mixing both would make the half you were after harder to reach. The popup appears only for what you type: loading a definition back into the field with *Edit selected* fills it from the program, and a popup dropping over a formula you asked to edit would be in the way.

Once created, the parameter behaves exactly like a channel read from the file. It appears in PARAMS, in ISO, in the INSPECT tables, in the CSV export, and it can be referenced by other formulas. *Edit selected* loads a definition back into the fields; the name field is disabled while editing, because renaming would orphan every expression that referred to it. Applying an edit re-evaluates the parameter everywhere. *Remove selected* removes it, and also removes everything that depended on it, saying which: *Removed MACH_SQ, and ENERGY which depended on it*.

The *Help* button opens the complete syntax reference, which is also reproduced in Chapter 10.

4.4 DIFF — comparing two runs, and comparing two files

The tab has two halves, and they answer two sizes of the same question. *Difference*, at the top, draws one channel's difference between two runs you choose — the rest of this section. *Whole files*, below it, compares two entire files against a table of thresholds and reports everything that moved too far; that is Chapter 5.

Two runs that look alike at tile scale can differ by three per cent somewhere in the middle, and the eye cannot find where. The first half puts that difference where it belongs: on the y axis, at the x it happened.

Choose a parameter and two runs. The tile then draws $A - B$ against the tile's own x axis, which is chosen here as well — the same setting PARAMS shows, writable from either, so asking this one question needs no other tab.

One parameter and two runs, and no more. The question a difference answers is "where do these two part company, and by how much", and a tile holding four differences of three channels answers nothing; a second difference is a second tile.

Both lists offer every run of every open file, whether or not it is ticked in DATA. A difference is a question about two particular runs, and having to tick them first — which would also draw them — is a step in the way of the question. The two may come from different files, which is the case this is most often wanted for: the same manoeuvre from two builds of the model.

The runs keep their own instants

Nothing says two runs were recorded at the same times, and runs from two files almost never are. So the difference is taken on *run A's* x, with run B read at those same x by linear interpolation, over the stretch of x the two have in common. Outside that stretch there is nothing to subtract, and extrapolating would draw a difference that was never measured — so the curve simply stops. Run A is the reference in both senses: its instants, and its extent.

Where SYNC is on, both runs are aligned on their event before the subtraction. That is what makes the comparison possible at all for the same manoeuvre flown at two different clock times.

While a tile is differencing, the parameters ticked in PARAMS are left alone and not drawn: DIFF is the more specific instruction about what the tile is for. *Stop differencing this tile* gives the tile back to them. Colour, dash, width and marker come from STYLE as usual, keyed on the channel, so a difference of `ALPHA` is drawn in the colour `ALPHA` has.

The tile is titled `ΔALT between two runs = f(TIME)`. The legend row takes two lines — the channel, then the two runs, with their files when the two differ — because on a tile a quarter of a page wide, all of that on one line runs past the plot it is labelling.

It reads four channels and no more: x and the parameter, from each of the two runs. They are collected into the same batched, multi-threaded preload as everything else on the page, so differencing a run of a 3 GB file over a network share costs two column reads rather than a walk of the file.

Nothing is drawn when the two runs share no x at all, when one of them does not carry the channel, or when A and B are the same run. The panel says which of those it is, because a difference that cannot be drawn looks exactly like one that is drawn and happens to be zero.

4.5 ISO — filtering runs by a value they hold constant

An ISO parameter is one whose value is constant within a run but differs between runs: the initial mass of a sweep, a Mach number, a configuration index. The ISO tab filters the runs by those values, and several ISO parameters can filter at once, combining with a logical AND so that each one narrows the set further.

Choose a parameter in the *Filter by* combo, which lists only parameters not already filtered on, and press *Add*. The parameter appears as a checkable node whose badge summarises the selection — *all 75*, *none of 75* or *2/3* — followed by *paused* when the filter is switched off, *grouped* when it is grouped, *not in the loaded files* when no open file carries it, or *N outside the groups* when some values fall outside every interval. Its children are the individual values, or the interval names once grouped, each separately checkable.

The tab is a **tree**: one collapsible node per parameter over its own values, so every filter and everything selected in it is on screen at once. It replaced a list of parameters above a list of the selected one's values, which could only ever show one filter's values at a time — with three active there was no way to see what was selected without clicking through them and remembering.

```
v MACH      3/12
    0.70
    0.75
    0.80
> WEIGHT    all 5   grouped
> INITIAL_MASS   2/3   paused   grouped
```

A parameter's checkbox is a **select-all over the values beneath it**. Ticking it ticks every value — or every interval, when the parameter is grouped — unticking it unticks them all, and it shows a partial tick when some but not all are on. That is the same thing the *All* and *None* buttons do to the selected parameter, reachable from the row itself.

The filter's own on/off flag is on the row's right-click menu, as *Use this filter*, where it is named rather than guessed at. Switching it off keeps the values you ticked — which is what makes "with and without this" a two-click comparison — and the row says *paused*, so a filter that is not narrowing anything still accounts for itself in the tree.

Four buttons act on the selected parameter: *Group…* opens the interval editor, *All* and *None* tick or untick every value, and *Remove* stops filtering by it. The context menu carries the same operations, plus *Ungroup* and *Use this filter*. Ungrouping keeps the intervals in state, so re-grouping returns to what you had set up.

Ticking an individual value also switches the filter on, since choosing what to show is a statement that the filter should be applied. Unticking everything does not switch it off, and does not switch it on either: it says nothing about whether the filter is paused, so a paused filter stays paused rather than being activated to admit nothing.

4.5.1 Grouping values into intervals

The header of the grouping dialog states how many distinct values there are and the range they span. The *Groups* spin box is bounded above by the number of values — there can never be more groups than values — and *Distribute evenly* splits the observed range into equal intervals, discarding boundaries edited by hand. The table is editable in its *Name*, *From* and *To* columns; *Values* is read-only.

The editing model is the reason overlapping intervals are impossible rather than merely rejected. N groups are stored as $N + 1$ sorted edges, so the *From* of one row and the *To* of the row above are the same number; editing either moves that single edge, clamped between its neighbours. Interior gaps and overlaps are therefore unrepresentable. The two outer edges may

still be moved inward, which is how an outlier is deliberately excluded, and the note beneath the table then reports how many values fall outside every group. A comma is accepted as a decimal point, and a cell that cannot be parsed silently restores its previous value. Changing the group count clears hand-typed names and re-splits evenly; names you typed yourself are never regenerated.

Bounds are inclusive at both ends and the first claiming group wins, so a value sitting exactly on a shared edge belongs to the earlier interval.

4.5.2 Two constraints, composed

The decisive property of this panel is what it does not do: applying a filter never writes into the DATA selection. A run is drawn if and only if

$$\text{ticked in DATA} \ \wedge \ \text{admitted by every active ISO filter.}$$

Ticking a run whose ISO value is excluded therefore puts it on no tile, and changing an ISO filter never re-ticks the DATA tab. The panel's summary line says how many of the DATA tab's runs are being held back, and the status bar repeats it.

Two controls, two pieces of state

The filter result is never written into the run selection. Were it, applying a mass filter would silently re-tick the DATA tab and re-ticking a run in DATA would appear to widen the filter: two controls over one piece of state, each overwriting the other's answer. The intersection is computed only where curves are actually drawn, and memoised, because every tile asks the same question on every rebuild.

Two details of the admission rule are worth stating. A run whose value for the parameter is not a number is never admitted — it has no value for that parameter, so no interval can claim it. And for an ungrouped filter, a value absent from the tick map counts as on, so a value that appears only after a new file is opened is shown rather than silently hidden.

4.5.3 What a workspace keeps, and what it does when the data has moved on

The filter definitions are saved: which parameters are filtered on, their intervals, and which values are ticked. What is not saved is the answer — which runs those filters admit describes the files the last workspace had open, and is rebuilt once the new ones are known.

A workspace outlives the run it was made from. The solver is re-run with fewer channels, a file is replaced, a parameter is renamed; none of that may raise, and none of it may quietly empty the plot. Three cases are distinguished, and the load reports each of them in one dialog titled *Workspace loaded with differences*.

Situation	What happens	Shown as
No open file carries the parameter	The filter constrains nothing: it cannot speak about a channel that is not there.	*not in the loaded files*
The parameter is there, but no run falls in a ticked interval	The filter genuinely admits nothing, and the plot is correctly empty.	*no run satisfies it*
The column is there but a run's value is not a number	That run alone is excluded — the column exists and this run has no value for it.	greyed in DATA

A filter with nothing to say says nothing

The first row of that table is the one worth dwelling on. Reading it the other way — a filter on a parameter a file does not carry excludes every one of that file's runs — is defensible in the abstract and wrong in every case it actually arises: it would make a filter on a parameter only some files carry hide every run of the others, and it would make a workspace whose ISO parameter has since disappeared open to an empty plot with nothing at all to explain it.

Entries that cannot be read at all are skipped individually rather than failing the load: a value that is not a number, an interval with non-numeric bounds, an entry that is not an object. A reversed interval is normalised rather than refused, and a grouped filter whose intervals did not survive is demoted to ungrouped rather than left in a state the tab would have to invent intervals for.

Very long value lists are truncated in the tree at four hundred children, with a non-checkable row reading *…and N more, still filtered on*. The ticks for those values live in the state, so the filter still applies to them; only the rows are omitted, because laying out several thousand tree items is slow enough to be felt. A parameter's checkbox acts on *every* value, including the ones past that cap.

4.6 STYLE — colour, line, marker and legend

The panel opens with four per-tile switches — *Legend*, *Grid*, *Log Y* and *Points* — whose defaults are on, on, off and off. *Points* draws a circular marker on every curve of the tile that has no marker of its own; a per-curve marker overrides it.

4.6.1 Two encodings, chosen separately

Two choices, made independently on each tile: *Colour by* sets the colour, and *Style by* the dash pattern and marker. A curve therefore carries two identities, so a tile showing the same three channels from three files can distinguish the channels *and* the files.

Colour by — *Parameter*, *File*, *Simulation* or *ISO* — sets the colour and nothing else. *Parameter* is the default, and leads the row: a tile is nearly always asked which quantity a line is, and colouring by run answers a different question — one that a twenty-five-run study answers with twenty-five colours that name nothing the user chose. Each radio is identified by its mode rather than by its place in the row, which is also what a workspace stores, so the row can be reordered without re-labelling a saved tile. Choosing *ISO* reveals an *ISO parameter* combo. If the tile has no ISO parameter yet, the combo displays the default without writing it, and selecting the radio commits that choice immediately — so that the table below is built from the same parameter the plot will use. Getting this wrong was a real defect: the table was built from the old choice and showed the wrong rows until something else forced a refresh.

Style by — *None*, *File*, *Simulation* or *ISO* — sets the dash pattern and the marker. Under *None* the dash and marker follow the colour key. Choosing *ISO* reveals a second combo, *Style ISO*, which need not name the same parameter the colour does.

The pairing this exists for is several files of the same parameters on one tile: *Colour by Parameter* and *Style by File*. The colour says which quantity, the dash says which file, and the two read at once.

Colour by File is the same question answered the other way round, for a tile where which file a line came from is the first thing to see rather than the second: every curve out of one result file takes one colour, however many channels and runs it carries. It keys on the file's

path rather than its name, so two studies both called `results.aoe` in different directories stay apart — the same key *Style by File* uses, so the two modes cannot disagree about what a file is.

Two modes on the same identity say one thing twice

Nothing stops *Colour by File* being chosen alongside *Style by File*, and the result is legible but wasteful: both dimensions carry the same information and the tile has nothing left to distinguish the channels with. The useful arrangement is one against the other.
It is also worth ticking *File* among the legend parts when colour keys on the file. The legend merges rows whose text is identical, so with only the parameter shown, the same channel from two files becomes one row — and the colour that distinguishes them goes unexplained.

Colour by	One colour per	Key
Parameter	y channel of the tile (the default)	`param::<name>`
Simulation	plotted run	`sim::<run key>`
ISO	value or interval of the chosen ISO parameter	`iso::<param>::<label>`
File	open file, whatever it holds	`file::<path>`

Style by	One dash per	Key
None	—	the colour key above
File	open file	`file::<path>`
Simulation	plotted run	`sim::<run key>`
ISO	value or interval of the *Style ISO* parameter	`iso::<param>::<label>`

The two key namespaces are the same, which is deliberate: a tile that colours and styles by the same thing gets the same key from both, and a curve keeps one identity rather than two that could drift apart.

Apply encoding to all tiles copies all four choices — both modes and both ISO parameters — onto every tile of every page, hidden tiles included.

Dash patterns are handed out in order of first use from `solid`, `dash`, `dot`, `dashdot`; past the fourth distinct identity the cycle repeats and a marker is added from the ten shapes, so a fifth file is still told apart from the first.

4.6.2 What decides a curve's colour

A colour, once allocated, is kept for the session. The palette has twelve entries and the registry hands them out in order of first use, so the first three curves on a tile are always red, then blue, then dark green, whatever the colouring mode — allocation depends on the order keys are first seen, not on what they describe. Three curves is the common case, and a set of figures whose three colours mean the same three things throughout is easier to read than one whose first colour depends on how many curves happened to be on the tile. There is deliberately no cyan near the front: cyan and a light green are the same colour once a line is one pixel wide. The remaining entries are ordered for separation. Loading another file therefore never recolours what is already on screen.

Re-assign palette deliberately forgets the current assignments so the next redraw allocates afresh, and *Reset widths* drops per-curve widths so all follow the global setting. Two global spin boxes set the line width, from 0.1 to 10.0 with a default of 1.6, and the marker size, from 1.0 to 20.0 with a default of 6.0.

4.6.3 The two tables

The *Colour* table and the *Line* table have different row sets: three parameters and four files is three colour rows and four line rows, not twelve of anything. Both scroll when the list is long.

Colour has one row per coloured identity in the current *Colour by* mode, with a swatch that opens a colour dialog. *Line* has one row per styled identity in the current *Style by* mode, with a dash combo over `solid`, `dash`, `dot` and `dashdot`, and a marker combo over eleven shapes from *none* to *hexagon*. With *Style by* set to *None* the Line table shows a single unselectable row reading *Style by is None — dash follows colour*; when nothing is plotted, both show *Nothing plotted yet*.

4.6.4 Setting several rows together

Rows are selectable, and a change made on one of several selected rows is written to all of them. Select by the names in the first column — click, then Ctrl +click or Shift +click — then pick a colour, a dash or a marker on any row in the selection.

A sweep of twenty runs wants the same dash on twenty rows far more often than it wants twenty different ones, and doing that one row at a time is twenty dialogs.

A change lands where it was made

Changing a row that is *not* in the selection changes that row alone. The alternative — treating any selection as the target — means a selection left over from an earlier edit silently repaints curves the user is no longer looking at, with nothing on screen to explain why.
The whole selection is written in one pass and one redraw is asked for, not one per row.

4.6.5 The legend

A legend row is assembled from up to four parts, each switched separately: *Parameter* (on by default), *ISO*, *File* and *Simulation*, joined by vertical bars. The dash pattern and marker are never spelled out in words, because the legend already draws a sample of the curve.

With *File* and *Simulation* off, rows whose text is identical are merged. That merging is the point of the switches: twenty-five runs sharing three ISO intervals produce three rows rather than twenty-five saying the same thing.

Ticking *ISO* writes every ISO identity the curve carries, not only the colour's. A tile colouring by parameter and styling by ISO therefore reads `ALT | MACH = 0.75`: one row per value, ordered by value — or by interval, when the parameter is grouped. When *Colour by* and *Style by* name the same ISO parameter it is written once, not twice.

The ISO value has to be *in* the text for the rows to survive, because rows with identical text are merged — which is what turns a fifty-run sweep into one row per value rather than fifty saying the same thing.

Beyond sixty rows the legend stops adding them and appends a pen-less entry reading *…and N more (legend caps at 60 rows)*. The cap is what keeps a large study viewable: pyqtgraph lays out two graphics items per row, and a thousand-run study froze the application here before it existed. The row names the cap rather than only counting what it left out, because a bare count reads as missing data — every curve those rows would have named is drawn.

What the cap counts is rows, not curves. A row the legend already holds — the same text drawn in the same colour, dash and marker — does not cost a second one, so one parameter taken from two files and labelled by file is two rows whether each file holds three runs or three hundred.

The *Corner* combo snaps the legend to any of the four corners. Dragging the legend on the plot puts it anywhere, switches the corner to *custom* and keeps the position, including

in a saved workspace. *Apply legend to all tiles* copies this tile's whole legend configuration — whether it is shown, its corner or dragged position, and which parts make up a row — onto every tile of every page, and reports how many tiles changed or says that every tile already had it.

4.6.6 Titles and axis captions

The captions beside Y and under X are blank. The tile's title names every selected parameter and the x channel, as `ALPHA, NZ = f(TIME)`, and a long list wraps onto a second line rather than being truncated.

A caption beside the axis would repeat what the title already says and spend a strip of a tile that is often one ninth of a page. The unit reaches the reader through the tick values and the INSPECT tables instead.

The caption is *hidden* rather than set to an empty string, because pyqtgraph shows an axis label whatever text it is given — an empty one still reserved the whole line it was meant to give back.

How much room the tick numbers get

pyqtgraph sizes the bottom axis from the text it drew the last time it painted, which is one paint behind: the height reserved is right for the font the axis had *before*, so the first layout after the tick font changes — every change of interface scale, and the first paint of every tile — reserves the previous font's height. Where the new font is taller, the numbers are drawn below the space reserved for them and the tile clips them.

It showed on Linux and not on Windows because the default font differs: DejaVu Sans at the same point size is taller than Segoe UI, so the figure pyqtgraph starts from is too small there and close enough to right on Windows.

So the height is not left to it. It is computed from the font's own metrics — the full line height including descenders, plus the tick offset, plus two pixels of air at the current scale — and set as a fixed height, which is right on the first paint, on every platform and at every interface scale. The left axis is still pyqtgraph's own, because its width is measured from the tick strings actually drawn, which it does get right.

A tile with nothing to draw says so in the same place: *empty tile*, *no parameter selected (x: TIME)*, *ALTITUDE (no x axis selected)*, or *3D tile — choose X, Y and Z*. A three-dimensional tile reads `ALTITUDE = f(EAST, NORTH)`.

4.7 SCALE — axis ranges and cross-tile constraints

While an axis is unlocked, its two boxes follow the plot as you pan and zoom, so ticking *Lock X range* or *Lock Y range* is a one-click way to freeze exactly what you are looking at. Locked, the boxes are the range: they are enabled, and typing in one moves the frame. The boxes span $\pm 10^{12}$ with six decimals and do not commit per keystroke — a value takes effect on Enter or when the field loses focus. A minimum typed above its maximum is normalised rather than allowed to blank the plot.

A locked range is a hard bound, not a starting point: the view's limits are set and per-axis mouse interaction disabled, so no drag or wheel can leave it. A degenerate range is refused, because a view frozen at zero width looks exactly like a crash.

The two *Apply* buttons attach a range to the variable rather than to the tile. *Apply X range to every tile with the same X axis* gives every plot against that x channel, on every page, the same horizontal scale, so they can be compared directly. *Apply Y range to every tile sharing a parameter* does the same for any tile showing any of this tile's y channels. Because these constraints are

re-applied on every redraw, they also govern tiles that acquire the parameter later — which is the whole point, and which the first implementation got wrong by walking only the tiles that existed when the button was pressed.

4.7.1 When two constraints meet on one axis

A tile has one y axis and may draw several channels on it, so it can be asked for two different vertical windows at once. Consider three tiles: the first draws `ALTITUDE`, the second draws `ALTITUDE` and `MACH`, the third draws `MACH`. Constrain `ALTITUDE` from the first tile and `MACH` from the third, and the second tile now carries two incompatible demands.

No arithmetic satisfies both, so none is invented. Parameters that ask for the same window are applied normally. Parameters that disagree leave the tile exactly as you had it — free, and therefore auto-fitting to show all of its data — and the SCALE tab says so, naming the parameters and their ranges, with the offending entries marked in the *Constrained variables* list. A button, *Use the range that covers both*, then offers the compromise as something you choose rather than something you are given; it locks this tile, not the variables, so the tiles whose scales the constraints were set to fix keep them.

Why not simply take the union?

Taking the smallest window containing both is the obvious answer. It is wrong twice over. A union is a superset of both ranges and therefore honours neither, which defeats the entire purpose of the feature: the point of constraining a variable is that two tiles showing it can be compared directly, and under a union they cannot. And when the two ranges differ in magnitude the union destroys the smaller channel — `ALTITUDE` at $[0, 13000]$ unioned with `MACH` at $[0, 1]$ leaves Mach occupying 0.008 % of the axis height, a flat line along the bottom edge, with nothing on screen to explain it. Which parameter survived depended on its magnitude rather than on any decision the user had made.

The x axis cannot fail this way. A tile has exactly one x channel, so the lookup returns at most one range however many x constraints are declared; the conflict is not representable. The same channel may carry one window as an x axis and a different one as a y axis, which is correct — they are different axes — and the *Constrained variables* list labels every entry with the axis it belongs to.

Three-dimensional tiles are skipped entirely. Their frame is the projection's own centre and zoom, and the two-dimensional ranges describe data extents that mean nothing to a projection; locking them did no visible harm but left a tile mysteriously locked if it was ever switched back to two dimensions.

The *Constrained variables* list shows every such constraint, and *Clear all variable constraints* removes them while leaving per-tile locks alone. *Unlock both and auto-fit* releases this tile and also pops its parameters out of the variable-wide constraints — without which the next redraw would simply re-lock it.

When the frame moves

A tile is fitted once, when it first acquires data. After that the frame moves only on *Fit*, on a double-click, or when SCALE changes something. An ordinary redraw — restyling a curve, ticking a run, toggling the grid — never moves the frame. Locking x on a tile whose y is free is the one exception: the y axis is re-fitted inside the new x window, computed by a single pass in the core that understands a non-monotonic x, because otherwise a windowed manoeuvre sits as a flat line at the bottom of the tile.

4.8 SYNC — aligning the runs on an event

A channel logged over several runs of the same manoeuvre carries that manoeuvre at a different place in each run, because each recording started at a different moment. Plotted against `TIME` the curves are the same shape smeared sideways, and comparing them means comparing the shapes rather than the clocks.

This panel shifts each run along the x axis so that a chosen event lands in the same place for all of them. Nothing in the file changes and no sample is altered; only where the curves are drawn.

The event

Condition	What it finds
first rises above	the first sample past a level you give
first falls below	the same, downwards
first crosses	either direction, judged against where the channel started
first starts to rise	no level needed: the first move of 2% of the channel's own range away from its first sample
first starts to fall	the same, downwards
reaches its maximum	the largest sample of the run
reaches its minimum	the smallest

The two *starts to* conditions take a threshold in percent of the channel's own span rather than an absolute level, which is what lets the same setting mean the same thing on a speed in m/s and an angle in radians.

Where it goes

Move the event to	Result
zero	the axis reads as time since the event
the first run's event	everything moves onto the first run drawn, which keeps the original axis values meaningful
a value I choose	for lining two tiles up on one reference

A car accelerating from rest, logged five times: choose `SPEED`, *first rises above*, `0.5`, and *zero*. Every run is then drawn with $x = 0$ at the moment it began to move.

What it does with the awkward cases

Three decisions worth knowing

The event is interpolated, not rounded to a sample. Two runs logged at different rates would otherwise align only to the sampling interval, which is visible as a step between them. Where the crossing itself falls inside a gap — `NaN`, which is what an unreadable token becomes — the first readable sample past the level is reported instead, because there is nothing to interpolate across.

A run in which the event never happens is left where it is, and counted. The panel reports "7 run(s) aligned, 1 left unaligned because the event was not found in them", and

List the events names each run with the value found in it. Hiding such a run would lose data; shifting it by a guess would be a lie.
Markers and bands are evaluated in each run's own axis. They go on meaning "this instant in this run" and travel with the curve they mark, rather than being reinterpreted as an offset from the event.

The alignment belongs to the tile, so one page can hold the aligned comparison beside the raw one. *Apply to every tile on this page* copies it across. The title says so as well: `ALT = f(TIME)  [0 = SPEED event]`.

It is saved in the workspace. A workspace written before this existed carries none of these fields, which the loader reads as the default — off — so it opens exactly as it was saved. The schema version is unchanged, deliberately: bumping it would make the workspace unreadable by the version that wrote it.

4.9 3D — three channels against each other

Ticking *Draw the active tile in 3D* turns the active tile into an orthographic projection of three channels, filling in plausible axes if the tile has none. The three combos set x, y and z; a 3D tile uses exactly one y channel. *Rotation* sweeps the azimuth from 0 to 359 degrees and *Tilt* the elevation from -89 to $+89$, clamped rather than wrapped because past straight up or down the projection turns inside out. Four presets — *Iso*, *Top*, *Front*, *Side* — set the pair directly, and *Apply these axes to every 3D tile* copies axes and view onto every other tile already in 3D.

Two switches control the depth cues. *Orientation axes (X, Y, Z)* draws the fixed-size triad in the corner, coloured by the universal convention — x red, y green, z blue — and hovering it names the channel on each axis and its range. *Reference grid and tick values* draws the back-wall grid and its numbers; without them the projection has no depth cue and no numeric scale, but on a very dense trajectory they can be turned off.

Each channel is normalised independently onto $[-0.5, 0.5]$, so three channels with unrelated units stay readable together; a channel that never moves collapses to the middle of the cube, which is the honest picture of a constant. In Pan mode a drag turns the object, middle-drag or Shift+drag slides it and right-drag or Ctrl+drag zooms; in Zoom mode the wheel zooms. Double-click or Ctrl+0 frames everything again.

Both encodings apply here. Colour and dash are resolved exactly as on a two-dimensional tile, so *Style by File* distinguishes the trajectories of two files in one cube. A projection has no legend, so the legend switches do nothing on a 3D tile.

Markers and areas are not drawn on a 3D tile

A vertical line is not a plane, and the indices a marker dot would use refer to the thinned, projected arrays rather than to samples. Annotations are therefore restricted to two-dimensional tiles.

4.10 INSPECT — reading values off a curve

Switch the mouse to Inspect and click a curve. The header names the file, the run and the x value at the click; the *At a point* tab then lists every channel of that run at that sample, with its unit, printed to six significant figures and with a dash where a value is not a number. *Copy to clipboard* writes the header and one tab-separated line per visible row — visible meaning after the channel filter, which accepts the same wildcards as PARAMS.

The nearest curve is found in pixels, not in data units. The two axes carry unrelated scales, so a distance measured in data space would always pick the channel with the larger numbers.

The *Statistics* tab gives minimum, maximum, mean, standard deviation and sample count for every combination of plotted run and y channel of the active tile. By default it reports over the visible x window and follows pan and zoom; *Whole channel instead of the visible range* widens it. Its header states the scope exactly, including the mixed case that arises when some runs have an x channel shorter than their y channel and therefore cannot be windowed at all. Both tables copy to the clipboard tab-separated.

Dragging in Inspect mode measures a gradient instead. A dashed line follows the pointer with the running figure beside it, and on release the panel reports the slope, the two deltas, both endpoints and the inverse. A purely vertical drag reports the change in y over no change in x rather than an infinity. A drag shorter than six pixels is treated as a click, so a slightly unsteady hand still inspects a point.

4.11 AREA — shaded bands

Add places a band across the middle of what the active tile currently shows — from 40 % to 60 % of the visible x span — using that tile's x channel. The editor beneath sets visibility, the x channel, the two bounds, the colour and the opacity, which runs from 0.02 to 1.0 with a default of 0.20. The bounds are normalised on commit, and the two buttons *Show all* and *Hide all* act on every band at once.

Bands are global: they appear on every page, on every tile whose axes can express them. They are drawn *over* the curves, because a band underneath them lost its edges wherever a curve crossed one.

In Select mode the edges of a band can be dragged directly on the plot, and the panel follows. There is deliberately no caption field: a caption typed here rebuilt the tile on every keystroke, deleting the very item being edited. Name a band with a TEXT note placed over it.

4.12 MARKERS — an instant, drawn two ways

A marker is a value of one channel — `TIME = 707.35`, say. *Add* places one at the midpoint of the active tile's x window. The editor sets visibility, the channel, the position, the colour and the dot size, which runs from 2.0 to 30.0 with a default of 9.0.

The dual rendering is the feature. On a tile whose x axis is the marker's channel the marker is a dashed vertical line, draggable in Select mode. On any other tile it becomes a dot on each curve, at the sample where the condition holds — which says something a vertical stripe could not: on a plot of angle of attack against altitude, where the aircraft was at that instant.

A marker that falls outside a run's range is refused rather than clamped to the last sample, because a clamped dot would silently claim the condition had been met. The band's equivalent rendering redraws the matching stretch of every curve in the band's colour, with separate passes through the band left unjoined.

4.13 TEXT — captions on a tile

Captions belong to the active tile — the deliberate opposite of markers and areas — and are positioned as a fraction of it, from 0.0 to 1.0 in each direction with $y = 0$ at the top. *Add text* appends a note reading *Note*, staggered downward from any notes already there. The editor sets visibility, the text (several lines are fine), the position, the point size from 4 to 72, bold, and the colour, which starts as the current theme's text colour. Nine *Place* buttons are corner and

edge presets. *Copy to all tiles* deep-copies this tile's captions onto every visible tile of every page.

Typing does not commit on every keystroke: a 400 ms idle timer fires after you stop, or the caption commits when the field loses focus, or immediately on *Update text now*. Committing per keystroke would flicker the tile and steal the caret. Notes can also be dragged directly on the plot in Select mode, and the spin boxes follow; clicking outside every note passes the click through, so empty space still selects the tile.

Chapter 5

Comparing two files

A run-to-run difference answers one question about two runs you already suspect. This answers a different one, about everything at once: given two whole files — yesterday's build and today's, the reference case and the candidate — has anything moved further than it was allowed to, and where?

The shape of it is four steps. You name the two files and a configuration that says which channels matter and by how much they may differ. The program checks that the two files can be compared at all, compares every matched run of every channel the configuration names, and puts what it found in a report window. Nothing is plotted until you ask for it on a line of that report; when you do, it generates pages of tiles that overlay both files with the crossings shaded.

All of it is in the lower half of the DIFF tab, and all of it is saved in the workspace.

5.1 Setting it up

File A and **File B** are two of the files open in DATA. File A is the reference: differences are measured at its x values, and it is the one the other is read against.

Thresholds… chooses the configuration file. The extension is not checked — a `.txt` file with the right structure is accepted, which is what the people who maintain these tables tend to have. It is read the moment you choose it, so a file that is not one of these says so while you are still looking at the dialog, and the group names it declares appear immediately as a list of colour swatches.

Group colours is that list. Every group in the configuration gets a swatch, pre-filled from grey, red, orange and green in the order the groups appear, and clicking one opens the colour picker. These are the colours the report and the shaded bands are drawn in, and they are a *choice* — nothing in the program reads a group's name. A group called BLACK is an identifier that happens to be a colour word, and it is drawn in whatever you pick for it.

Run diff does the work. **Show report** reopens the report of the last comparison, at any time. Underneath, once pages have been generated, a combo lists them and goes to one.

Why every run is ticked before the comparison runs

A generated page is a page like any other, and it is read through the same DATA and ISO filters as the rest. A run left unticked three tabs ago would come out as a blank tile on a page you asked for because that run was flagged — which is the least helpful answer the program could give. So pressing *Run diff* ticks every run of every open file and admits every ISO value first. The comparison is about every run in the two files, and the filters are set to say so.

5.2 What must be true before anything is compared

Three things are checked, and each one is a separate message, because they send you to different places:

- the two files hold the **same number of runs**;
- their **run identifiers match one to one**, and no identifier appears twice in either file;
- the **same channels** are present.

If any of them fails, a warning says which and the comparison does not run. A non-regression check that half-ran and reported what it managed is worse than one that did not run.

Runs are paired by **identifier**, never by position. Run 3 of one build is often run 7 of the next — solvers finish in the order they finish — and a comparison that paired them by position would report every run as broken while telling you nothing.

5.3 The configuration

The file is YAML of one particular shape: a list of groups, each with a list of `algoconfs`, each of those naming the channels it covers, the threshold, and optionally the x axis to compare along.

```
- group_name: "BLACK"
  algoconfs:
    - variables: "ALFA"
      x: "TIME"
      threshold: 25
    - variables: "Q1;TETA;BETA"
      threshold: 10

- group_name: "AMBER"
  algoconfs:
    - variables: "ALFA"
      threshold: 10
    - variables: "Q1;TETA;BETA"
      threshold: 5
```

The rules:

- `variables` is split on semicolons, and each name becomes a rule of its own with the same threshold.
- `x` is the channel the comparison runs along, and defaults to `TIME`.
- `threshold` is a number, integer or decimal.
- **Keys this program does not know are ignored**, at any level and to any depth. That is what makes a real file — full of fields about things other than this — readable as it stands.
- If the same group and channel appear twice, the last one wins. These files are edited by hand, and a repeat is a correction.
- Blank lines, `#` comments, and the `---` and `...` document markers are skipped.

Only the channels the configuration names are compared. A rule set is a statement about which channels matter; comparing the other five hundred would turn the report into a list of everything that moved.

It is read without PyYAML

The shape above is narrow enough to read directly, and that is deliberate: the machines this runs on reach the package index one negotiated wheel at a time, and a feature that needed a new dependency would be a feature nobody could install. What is read is exactly the shape above. Anchors, flow collections, multi-line scalars and nested lists have no meaning in it and are refused rather than half-understood — a file that needs them is not

this file. A structural problem is reported with its line number.

5.4 What counts as a violation

For each matched run and each channel a rule names, the two series are read and subtracted, and a sample is a violation where

$$|A - B| > \text{threshold}$$

strictly: a difference exactly equal to the threshold is not a violation. A **flag** is raised for a (run, channel, group) when at least one sample violates it. A sample where either file has no number is not a difference anybody measured, and is skipped.

The two files need not be sampled alike. Where their x grids differ, File B is read at File A's x values by linear interpolation, over the stretch of x the two have in common. Nothing outside that stretch is compared and nothing outside it can raise a flag: extrapolating would report a difference that was never recorded. If the two share no x at all, that run and channel are reported as not compared rather than passed.

5.4.1 Priority, and what gets shaded

Priority is the threshold *value*: a higher number is the more serious group. Nothing else decides it — not the order the groups appear in, not their names.

A deviation large enough to cross a high threshold has crossed every lower one too, so a channel is usually flagged for several groups at once. The report lists all of them, worst first. The *shading* does not: at each sample only the highest-priority violated group is drawn, so the bands never stack and the colour you see is the worst thing happening there. Contiguous samples owned by the same group merge into one band.

The specification's own example: if RED (threshold 4) is violated on $[2, 4]$ and BLACK (threshold 6) on $[3, 5]$, the shading reads RED on $[2, 3)$ and BLACK on $[3, 5]$.

A group can be flagged and have nothing shaded

If a deviation is above the higher threshold everywhere it is above the lower one, the higher group owns every one of those samples and the lower has nothing left to draw. It is still listed in the report, because it was exceeded, and a report that hid it would be hiding a fact about the data. The band is a drawing decision; the flag is the measurement.

5.5 The report

The report opens by itself when a comparison finishes without an error, in a window of its own rather than in the tab. A non-regression result is read once, carefully, and then acted on or dismissed; a tab would put it behind whichever tab was opened next, while a window can be left open beside the plots it generated. It is not modal — the point of plotting a line is to look at the plots.

One line per run that raised at least one flag. Runs with nothing to say are not listed, and a clean comparison says so in one sentence.

```
RUN_001: ALFA(BLACK), ALFA(AMBER), BETA(AMBER), Q1(BLACK), Q1(AMBER)
RUN_004: ALFA(AMBER)
```

Each group name is drawn in the colour chosen for it in the tab. Long lines wrap rather than pushing out a horizontal scrollbar: a run flagged on eighteen channels has a line of thirty entries.

Every line carries a *Plot* button, and double-clicking the line does the same thing. **Nothing is plotted until you press it.** Forty flagged runs at sixteen tiles a page is not something to do on anybody's behalf; plotting is one explicit act on one line, and the window can be closed without generating anything at all.

Anything that could not be compared is listed beneath, named by run and channel: a missing x channel, two runs that share no x range, a channel that could not be read. A comparison reports what it skipped rather than passing it silently.

5.6 The pages it generates

Pressing *Plot* on a line generates pages named `DIFF (1)`, `DIFF (2)` and so on, each a 4×4 grid. One tile per flagged channel, in the order the report lists them — worst group first, so the first tile of the first page is the worst news. Sixteen channels fill a page and the seventeenth starts another; plotting a second run continues the numbering rather than starting again, so the pages of a comparison read as one sequence.

Each tile overlays the same channel of the same run as the two files recorded it, in two fixed colours, with the violated regions washed in behind the curves in their group's colour. The legend names both series on their own lines:

```
regression_a.aoe - RUN_001 - BETA
regression_b.aoe - RUN_001 - BETA
```

The two colours are fixed rather than taken from the palette, because on a page of sixteen overlays the question is always "which of these two is which", and the answer should not change from tile to tile. STYLE can restyle them like any other curve; zoom, pan, the scale locks, the annotations and the exports all work as they do everywhere else.

A generated tile names its *run identifier* and its two files, not run indices, and it carries its own bands. Three things follow from that. The shading is the verdict as computed, not something re-derived on every pan. A workspace reopened a month later shows the bands it was saved with, with no file to re-read and no configuration to re-parse. And the tiles draw whatever is ticked in DATA: the overlay was generated to answer one question about one run, and narrowing the filters afterwards must not blank the page that answered it.

Running a second comparison removes the pages the previous one generated, so one set of pages is never read as the answer to another question.

5.7 What the workspace keeps

Saved: the two files, the path of the configuration *and the rules read from it*, the group colours, the report, and the generated pages with their bands. Reopening restores all of it without recomputing anything.

The rules are saved as well as the path on purpose. A threshold table is maintained by whoever owns the thresholds and is edited in place; a report has to go on meaning what it meant when it was computed, not what the file on disk says today.

5.8 Trying it

`sample/` holds a pair built for this, generated with the executable (§2.2.1): `regression_a.aoe`, `regression_b.aoe` and `regression_thresholds.yaml`. Twenty channels, four runs under the

same identifiers in a different order in each file, the second file sampled more coarsely than the first so the interpolation is exercised, and deviations engineered in known places — one run clean, one flagged on eighteen channels, one on five, one on a single channel. Add both files in DATA, name them as A and B, choose the configuration, and press *Run diff*.

Two groups with the same threshold have no order

Priority is the threshold value, so two groups sharing one are tied and which of them gets shaded is not defined. The program does not pick for you: it warns, naming the channel and the groups, when the configuration is read and again in the report. Giving one of them a different threshold is the fix.

Chapter 6

Pages, grids and tiles

Duplicate copies the whole page, tiles and all: the copy is named `<name> (2)`, inserted straight after the original and made active, since a duplicate is nearly always the start of a variation on the page it came from. Duplicating a duplicate gives `(3)` rather than `(2) (2)`.

The **remove** button asks before removing a page, and says how many configured tiles are on it: it is one character wide and sits beside Add, where a mis-click would otherwise take a page of work with no way back.

Double-clicking a tile makes it active, fits it *and* brings PARAMS forward — three things from the gesture that means "this tile, now".

6.1 The page and tile bar

The bar immediately above the grid governs layout. On the left, a combo switches page, flanked by previous and next buttons, and a name field renames the current page on Enter or when it loses focus. Four small buttons add a page, remove the current one, and move it earlier or later in the order. Removing the last remaining page is refused.

In the middle, the *Grid* button drops a five-by-five matrix in the manner of a word processor's table picker. Sweeping the pointer highlights the layout and the caption underneath reads, for example, *2 x 3 (6 tiles)*; releasing chooses it. Cells whose layout would exceed twenty-five tiles are painted faint and cannot be chosen.

To its right, *Tile N* names the active tile and four arrows swap it with its neighbour to the left, above, below or right; each arrow is enabled only when that move is possible, and the active index follows the tile so the move is its own inverse. Then four symbol buttons — two offset sheets, scissors, a clipboard and a crossed-out frame — copy, cut, paste and clear the active tile's whole configuration. Pasting installs an independent deep copy, so pasting twice gives two tiles that can then diverge. This clipboard lives in the process only and is not saved to a workspace.

They were the four words until 2.3.0. As words they cost about 210 px of a bar that also carries the page controls, the grid picker and the tile arrows; as symbols they cost four square buttons, and they are the symbols every other application uses for them. The tooltips still say the words.

Last, behind its own rule, *Fit* (Ctrl+0) auto-scales every tile, honouring axis locks; double-clicking one plot fits just that one. Beside it, an open padlock takes every X and Y scale constraint off every tile on *every* page, releases every variable-wide constraint as well, and then fits everything.

Why the padlock reaches pages you are not looking at

Locks are set to make a set of plots comparable, and they are set on several tiles at once — often with *Apply to all tiles* in SCALE. A page left pinned behind the one on screen is then the next thing to look broken, and it looks broken in a way that gives no clue where to go and fix it.

Each tile's stored limits are kept, so a lock put back on by hand returns to the numbers it had rather than to zero and one. The fit afterwards is not decoration: an unlock on its own

changes nothing on screen — the axes stay exactly where the lock was holding them — so the button would read as doing nothing at all.

One press, not two

A lock is held in two places. The tile carries `lock_x` and `lock_y`, and the view box carries a hard bound — pyqtgraph's `setLimits` — which is what stops a drag or a wheel taking the view straight back outside the range the lock imposed.
The bound belongs to the view box, so it outlives the lock that set it. Clearing the flags and fitting in the same breath had the fit computed correctly and then clamped back to the range just released, and the bound was only dropped at the end of the fit — which is why a second press appeared to work where the first had not. The bound is now released before the ranges are computed, and re-applied afterwards for whichever axis is still locked.

Why it must release the variable-wide constraints too

A constraint declared for a *parameter* rather than for a tile is not held on the tiles at all. It lives on the application state, in `global_x_limits` and `global_y_limits`, and `apply_global_limits()` re-imposes it on any tile that acquires that parameter — which is precisely what makes such a constraint apply to tiles filled in after it was declared rather than only to the ones that existed at the time.
So clearing the tiles alone is not an unlock. It leaves every tile momentarily free and then locks the next one that picks the parameter up: declare a constraint, press the padlock, drop a parameter on an empty tile, and the tile arrives constrained. The button has to release both, and the status bar reports the two counts separately.

Fitting an *empty* tile frames nothing, and is not recorded as having framed it. That matters because double-clicking is also how PARAMS is opened, and on a fresh start the tile being double-clicked is empty: the gesture that means "I am about to put something here" would otherwise mark the tile as having a view worth keeping, and the first curve dropped on it would inherit the empty tile's range instead of being fitted.

Fit sits behind its own rule on this bar because it moves no tile and copies nothing: it re-frames what is already there. Its shortcut is registered on the window rather than on the bar, so it works from anywhere in the window — an action that is on no widget has no shortcut at all.

6.2 How a page is modelled

Every page always carries twenty-five tile objects regardless of its grid, and the visible count is the product of rows and columns, capped at twenty-five. Shrinking a grid therefore hides tiles rather than destroying them, and they return unchanged when it grows again. Only the visible tiles are written to a workspace; loading pads the list back up to twenty-five.

The tile widgets themselves are recreated only when the shape of the grid changes. Constructing a pyqtgraph plot widget costs tens of milliseconds, so a fresh 5×5 page costs most of a second; switching between two pages of the same shape reuses the widgets and merely redraws them.

6.3 What a tile remembers

Field	Meaning	Default
`x_param`, `y_params`	the channels drawn; a 3D tile adds `z_param`	empty
`show_legend`	legend visible	on
`legend_position`	corner, or `custom` once dragged	`top-left`
`legend_x`, `legend_y`	dragged position as a fraction of the view	0.02, 0.02
`legend_show_param`	the channel name appears in a legend row	on
`legend_show_iso`	every ISO identity of the curve appears	off
`legend_show_file`	the file name appears	off
`legend_show_sim`	the run's own identifier appears	off
`legend_show_style`	withdrawn; read for compatibility, never drawn	off
`diff_param`	the channel differenced between two runs; empty for an ordinary tile	empty
`diff_a`, `diff_b`	the two runs, as (file index, run index); A is the reference	$(-1, -1)$
`lock_x`, `lock_y`	the axis is frozen at the stored range	off
`x_min`, `x_max`	horizontal range, in data units	0.0, 1.0
`y_min`, `y_max`	vertical range, in data units even in log mode	0.0, 1.0
`view_initialised`	this tile has been fitted once already	off
`log_y`	logarithmic y axis	off
`show_grid`	grid lines	on
`show_points`	fallback circular marker on every curve	off
`color_mode`	parameter, file, simulation or ISO	parameter
`color_iso_param`	the ISO parameter that colours the tile	empty
`style_mode`	none, file, simulation or ISO	none
`style_iso_param`	the ISO parameter that dashes the tile	empty
`plot3d`	draw as an orthographic projection	off
`azimuth`, `elevation`	view angles in degrees	45.0, 25.0
`show_3d_axes`, `show_3d_grid`	triad and back-wall grid	on, on
`view3d_cx`, `view3d_cy`, `view3d_half`	3D pan centre and zoom half-width	0.0, 0.0, 0.95
`texts`	the tile's captions	empty

All of it is saved in a workspace. A workspace that carries no `style_mode` or `style_iso_param` opens with *Style by* set to *None*, under which the dash follows the colour key.

Why `legend_show_style` is still read

It is accepted from a workspace and never used by anything that draws: the legend shows a sample of the curve itself, dash pattern and marker included, so spelling those out in the text said nothing the row was not already showing. The field is retained so that a workspace written while it existed still loads.

The tile's title is generated from this state and states plainly when something is missing. With several y channels it names them all and wraps onto two lines; see §4.6.

Two 3D view fields deserve a note. The pan centre and zoom half-width are kept separately from the two-dimensional ranges, because those describe data ranges and mean nothing for a projection. The default half-width of 0.95 is chosen because the normalised cube's half-diagonal is $\sqrt{3}/2 \approx 0.866$, so 0.95 frames the whole box at any rotation with a small margin.

6.4 Robustness when the files change

When the set of open files changes, the application re-points anything that no longer resolves and reports what it did in the status bar — for example *page 2, tile 3: x axis VTAS -> TIME*. A missing z channel and a missing colour-by or style-by ISO parameter are cleared; a missing x axis falls back to the default. A missing y channel is deliberately left named: it plots nothing today and comes back when the file is reopened, which is friendlier when a study is loaded one file at a time.

With no file open at all, none of that happens. Closing a study does not destroy a layout, and reusing one workspace against another study — remove the old file, add the new one — keeps every tile's axes.

Closing a study leaves its computed parameters defined. That is deliberate: opening the next study should not throw away the formulas set up for it. It does mean that two different questions have to be asked separately.

`Session.parameters` answers "what names may the user write", and lists the file headers together with the computed channels — so it is *non-empty* even with nothing open. `plottable_parameters` answers "what can carry an axis", and is empty while no file is open, because a formula over columns that are not loaded evaluates to nothing. Everything that chooses a channel on the user's behalf reads the second, so that removing one study and adding another cannot re-point every tile's x axis at whichever formula sorted first.

A tile whose x channel is genuinely absent from the new study is still re-pointed, and the status bar still says so.

Chapter 7

Exporting, and saving your work

7.1 Image

Image writes the active page as a PNG, grabbing the plot grid widget exactly as displayed: every visible tile with its legend, markers, shaded areas and captions, at screen resolution, without the toolbar, rail or panels. The extension is appended if you omit it. This is the way to get a figure out of the program and into a report.

7.2 CSV

CSV writes the active tile's curves at full resolution, on a worker thread. The header row is `file`, `run`, `parameter`, the name of the tile's x channel, and `value`; note that the fourth column header is the channel's name rather than a literal. There is then one row per sample per combination of plotted run and y channel, in the order the runs are plotted and then the order of the tile's y list, with both numbers written to nine significant figures. Runs whose x or y channel is empty are skipped, and the status bar reports the sample count written.

Why the export is not what you see

The plot is decimated to roughly two points per horizontal pixel. The export is not: silently handing someone twelve hundred of their five hundred thousand points would be a trap. If the tile is empty or nothing is plotted, the button says so rather than writing an empty file.

7.3 Workspaces

A workspace is a single JSON file with the extension `.aoews`. It carries the list of open files by absolute path, the names you gave those files and their runs, every computed parameter, every ISO filter with its intervals and ticks, every page with its visible tiles and their complete state — both encodings included — the run selection, the whole style registry, the markers, the shaded areas, the variable-wide scale constraints, the arrangement of the tab rail, the file comparison with its thresholds, colours and report, the theme and the mouse mode. What it deliberately does not carry is any derived answer — which runs the ISO filters admit describes the files that were open when it was written, and is recomputed against whatever is open when it is read.

The schema is at version 3. Files also record `app_version`, which is the build that wrote them and is informational only.

7.3.1 New, Save, and Save as

New workspace (Ctrl+N) starts again from nothing: no files, one empty page, no filters, no comparison. The theme and the interface size are view preferences and stay as you had them.

There are two save buttons, not one with a modifier, because which file is being written is the thing that gets got wrong. *Save workspace* (Ctrl+S) overwrites the current workspace, asking for a path only if there is not one yet. *Save workspace as…* (Ctrl+Shift+S) always asks,

and then continues in the new file — leaving the one you came from exactly as it was on disk. That is how a variant is made.

New, *Load* and closing the window all ask the same question first, and only when there is something to lose: *Save*, *Discard* or *Cancel*. *Save* on a workspace that has no file yet opens the dialog, and cancelling *that* cancels the whole action — the one place where carrying on regardless would turn a save into a discard.

Three places say which file you are in. The window title carries its name, with a leading asterisk while there are unsaved changes. The *Save* tooltip names the file it would overwrite in full, or says that there is no workspace file yet and it will ask. And the status line after a save repeats the path and what went with it — *Workspace saved to C:\study\sweep.aoews (including 2 formula(s), 4 renamed run(s), 3 ISO filter(s))*.

7.3.2 Names given to files and runs

A file or run renamed in DATA (§4.1) comes back under that name when the workspace is loaded.

The names ride in two optional keys, `file_names` and `sim_names`. Both are keyed by identity rather than by position — a file by its path, a run by the core's `<file>::<id>::<ordinal>` key — because a workspace is routinely opened against files that have since gained a run or lost one, and a name pinned to "the seventh run" would then land on whichever run happens to sit there. A name whose file or run is not in the data is dropped without comment; there is nothing for it to label.

Runs are written only where the name differs from the identifier the file itself carries, so a thousand-run study does not write a thousand entries saying nothing. A file's name is always written, because the automatic one already depends on what else is open — two files of the same basename make the second *out.aoe (2)*.

Neither key is written when nothing was renamed, and a workspace carrying neither loads with each file and run under the name its own contents give it. They are extra keys rather than a schema bump for that reason: nothing older needs to understand them to keep working, and nothing newer is confused by their absence.

The names are applied on the worker thread, immediately after the files are opened and before anything reads a name, so the first paint already shows them.

7.3.3 Loading against data that has moved on

Loading reads the file, then does everything that touches the core on a worker thread: clears the session, opens each file, applies the saved display names, and re-creates the computed parameters. Any missing sidecars are started afterwards, in another process, once the workspace is on screen.

The formulas are installed in as many passes as it takes, repeating until a pass installs nothing new. A single pass is not enough: a JSON object has no dependency order, so a formula defined in terms of another would be refused — correctly, as an unknown name — and silently dropped. Repeating settles any acyclic chain, and what is left over genuinely cannot be defined against this data.

Nothing here stops the workspace loading. Whatever resolves is in place, and one dialog titled *Workspace loaded with differences* then lists what could not be: files that could not be opened, formulas that could not be defined and why, ISO filters on parameters none of these files carry, and ISO filters that no run satisfies.

JSON was chosen over an ad-hoc delimited format because such a format breaks the moment a file path or a parameter name contains the delimiter, and cannot be extended without a version bump nobody implements. The loader is tolerant by design: it sets only the

attributes it recognises and ignores the rest, which is how a workspace carrying a field this version has retired still loads, and how one that names no *Style by* opens at *None*.

What a workspace still does not carry

The active tile index, the tile clipboard, the memory limit, the window geometry, and the hidden tiles of a shrunken grid. The `"version"` key is written but nothing branches on it; the loader's tolerance of unknown keys is what stands in for version handling.

7.4 Background work, and why the interface freezes when it does

Anything that touches the core runs on a worker thread: opening files, building sidecars, loading a workspace, reading channels, reading every channel of a clicked run, and writing a CSV. This works because `ctypes` releases the interpreter lock for the duration of every call, so a Python thread sitting inside a core function genuinely runs in parallel with the event loop — while the core is using all of its own worker threads inside that one call.

The governing constraint is not the obvious one. The core session *is* internally synchronised: every C ABI entry point takes a reader-writer lock, so two threads calling it cannot corrupt it. What a lock cannot make safe is what the viewer does with the session, which is to hold pointers into its sample buffers and draw from them. Adding a file, reloading one or writing a sidecar moves those buffers, and a plot midway through reading one then reads freed memory — a lock around each call does not help, because the pointer was handed out before the call that invalidated it.

So the rule is that while a task runs the interface does not call the session at all: the panels, the rail, the page bar and the grid are disabled and the tiles show a placeholder. Tasks queue rather than overlap, and for redraw work only the newest queued request is kept, since an older one is by then worthless. There is no cancellation: a running task cannot be interrupted. While the interface is disabled the pointer shows a wait cursor and the progress bar counts whatever can be counted — see section 3.4 — because a frozen window that says nothing is indistinguishable from a crashed one.

7.4.1 Work that does not freeze anything: the sidecar builder

Writing a sidecar is the one long job that has been moved out of the application entirely. It does not fit the scheme above: it is a second full pass over the file, and the core holds its write lock for the whole of it, so as a worker task it blocked every read the window wanted and left the interface disabled from beginning to end. On a 2.9 GB HDF5 file of 50 runs and 1000 parameters, that was around five minutes between asking for the file and being able to use it.

Since 2.4.0 the application starts a copy of itself instead:

```
visimu --build-cache run_001.aoe run_002.hdf5
```

detached, with no console window on Windows, once the file is already open and drawn. That process has a session of its own, so there is nothing for the window to queue behind and nothing to disable; the only trace of it is the status-bar note. It is polled every 1.5 seconds, purely so that the note can be taken down when it ends, and the timer stops when no build is running.

The flag works on its own as well, which is the useful side effect: a directory of results can be warmed overnight so that nobody waits for a first open. Both kinds of file go through it — an AOE file needs the parse, an HDF5 file needs every column fetched first, which is exactly why doing it in-process reinstated the wait that reading HDF5 lazily removes.

What the builder is not allowed to do

It reads files it does not own, possibly on a share, possibly while a solver is writing them. So:

- It never writes to the source. It opens it read-only, and the only file it creates is the sidecar.
- It never holds a lock that would stop anyone else writing to the source. HDF5 takes one on open — even read-only — unless `HDF5_USE_FILE_LOCKING=FALSE` is set before the library initialises, which the reader does at import; the memory mapping used for AOE files is opened with `FILE_SHARE_WRITE`.
- If the source changes while it works, it abandons the sidecar and deletes it — through the core, which drops the mapping first, since on Windows a mapped file cannot be deleted. A sidecar mixing a file's before and after is the one failure that would put numbers on screen that were never in the file together.
- It never deletes or modifies the source, under any circumstance, including the one above.
- It fails quietly. A sidecar is an optimisation nobody asked for, and the file is already open and working without it.

Most redraws never reach a worker at all. Before dispatching, the application computes a signature of which channels each file must supply and which runs are plotted; when that matches what is already resident and no task is running, the grid is redrawn synchronously on the interface thread. Restyling a curve, toggling a grid or editing a limit is that kind of redraw, and routing it through a background task cost far more than the read it was meant to cover — starting a task disables and re-enables the panels, the rail and the whole grid, and every plot repaints on each of those transitions.

Chapter 8

The AOE file format

8.1 Overall shape

An AOE result file is plain ASCII text. There is no magic number, no preamble, no version field and no binary header; the only structure is a set of directive lines whose first non-blank character is an asterisk, with whitespace-separated decimal numbers between them.

```
*PARAMETRES
 'TIME'   'ALPHA'  'MACH'  'NZ'
*TABLEAU : SIM_001
 0.000 1.2340 0.7500 1.2500
 0.020 1.2350 0.7501 1.2499
*TABLEAU : SIM_002
 ...
*FIN
```

The parser recognises exactly two keywords, `*PARAMETRES` and `*TABLEAU` — note the French spelling of the first, without accent. A line is treated as a directive if and only if its first non-blank character is an asterisk; the keyword is then located by substring search within that line, which is why a form such as `*TABLEAU NUM SIMU: 1.2.0` is recognised as readily as `*TABLEAU : SIM_001`. Any other asterisk directive — `*FIN`, `*ENTITY`, a comment line — closes the open block and ends header reading. Nothing is required to terminate the file: `*FIN` is not special-cased, it is simply one more directive that closes the last block.

No magic bytes, and what that costs

Because nothing identifies the format, the only test of whether a file is an AOE file is whether parsing it yields at least one parameter. A file that maps successfully but contains no usable `*PARAMETRES` block is rejected; a file with headers but no `*TABLEAU` at all opens successfully with zero runs, because nothing rejects that case. Opening also fails if the file cannot be mapped, which deliberately includes a zero-length file.

8.2 Two header dialects

The `*PARAMETRES` block comes in two forms, and the parser chooses per line rather than per file.

The **packed** dialect puts several single-quoted names on one line and carries no units. Every non-empty quoted token on the line becomes a parameter, and its unit is the empty string.

```
*PARAMETRES
 'TIME'   'ALPHA'   'MACH'   'NZ'
```

The **indexed** dialect puts one parameter per line, prefixed by an ordinal and suffixed by a unit. The first quoted token is the name, the second — if present — is the unit, and anything further along the line is ignored.

```
*PARAMETRES
      1 'TIME' 's'
      2 'PARAMETER001'  ''
      3 'PARAMETER002'  'deg'
```

The test that distinguishes them is deliberately strict: digits, then only blanks, then a quote. A packed line begins with the quote itself and so can never match. The strictness matters more than it looks. Treating every quoted token on an indexed line as a name turns a 585-parameter file into 1156 parameters, puts `deg`, `ft` and `s` into the parameter list, and shifts every column index — so that asking for `TIME` silently returns another channel's samples. Correspondingly, quote extraction deliberately emits empty tokens, because the indexed dialect writes an absent unit as `''` and dropping it would shift the following line's unit onto the wrong parameter.

The header section is opened by the first `*PARAMETRES` encountered; a second block is ignored, which is what allows these files to repeat the header before every run. It is closed by whichever comes first: a line that yields no names once names have started, a blank line once names have started, or any asterisk directive.

8.3 How runs are delimited

A `*TABLEAU` line opens a run. Its identifier is everything after the first colon on the line, whitespace-trimmed; if there is no colon, or nothing after it, the identifier becomes `SIM_` followed by the run's one-based ordinal, without zero padding. The run's data begins at the start of the line following the marker, and its end offset is provisionally the end of the file and is rewritten the moment the next directive is met. That design — recording the end as the scan goes — is why the later parse tasks need neither a search nor a per-line terminator test.

Each run also carries a unique key of the form `flight_a.aoe::SIM_001::0`: the file's name, the identifier from the file, and the ordinal. Styles are attached to it and `sim_names` in a workspace is keyed on it, so it has to stay still under everything that is not an identity change.

Two things are not identity changes. Renaming a run does not touch it — the key is not derived from a run's display name. And renaming a *file* does not touch it either: the `<file>` part is the name the file was first given, which `Session` derives from the path before the file is parsed, not the label it currently carries.

Why the second of those had to be said out loud

The `<file>` part *was* the display name, and it went unnoticed because nothing rebuilt the keys. `reload()` does: it re-opens the dataset and builds every key again. So once reloading became something a user does on purpose (§3.5), renaming a file and then reloading it renamed every one of its run keys, and every colour, dash and marker chosen for those runs was left attached to keys nothing answered to.

8.4 Records, rows and wrapped lines

A logical record holds one value per header. Fortran writers wrap a record across several physical lines when it exceeds the format width, so the column counter carries across newlines and resets only after a full set of columns has been read. During the index scan the parser records how many values appeared on the first data line inside a run; only when that equals the column count is the file known to be one record per line, and only then may byte ranges be split at newlines for parallel parsing.

A data line is one whose first non-blank character is a digit, a minus, a plus or a full stop. Everything else inside a run's byte range — free text, blank lines — is skipped silently, and a skipped line does not reset the wrapped-record column counter.

Lines are split on the line-feed character, then trimmed on the right of any run of carriage returns, spaces and tabs, and on the left of whitespace. Both Unix and Windows line endings are therefore handled; classic Mac files, which use a lone carriage return, are not.

8.5 Numeric encoding

Values are decimal text, never binary. The reader accepts rather more than `strtod` would, because Fortran output requires it.

Form	Example	Notes
Plain	`1.5`, `-0.002`, `+3`	leading sign accepted
C exponent	`1.5e3`, `1.5E-3`	
Fortran double exponent	`1.234D+05`, `1.234d5`	`D` and `d` are read as `E`
Fortran bare-sign exponent	`1.234+05`	accepted only once mantissa digits have been seen
Non-numeric token	`NaN`, `Inf`, `*****`	becomes a quiet NaN — never zero

The last row is a design decision worth stating plainly: an unreadable value becomes not-a-number, so a gap in the data leaves a gap in the curve, rather than a spurious zero that would silently distort both the plot and any statistic computed over it.

Mantissa digits accumulate into a 64-bit integer up to nineteen significant digits with the decimal exponent tracked separately, and are scaled once at the end against an exactly-representable table of powers of ten. Digits beyond the nineteenth increment the exponent in the integer part or are dropped in the fractional part. If an exponent marker turns out not to be followed by digits the pointer is rewound, so `1.5e` reads as `1.5` and leaves the `e` to the caller. The result is computed as a double and narrowed to a float for storage.

8.6 File extensions

The core is entirely extension-agnostic: it takes a path, maps it, and parses the content. The only place a suffix appears is the cache-path builder, which appends `.aoec` to the full source name. The extension list lives solely in the file dialog's filter, which accepts `*.aoe`, `*.dat`, `*.txt`, `*.res` and `*.out`. Those are the same format under different customary names; the two real dialects are the packed and indexed header forms, and they are distinguished by content rather than by filename.

8.7 Files that are not AOE: HDF5

A result file stored as HDF5 opens like any other. The channels, the runs and the samples are read; everything after that — decimation, ISO filtering, computed parameters, styles, workspaces — is the same code on the same data, because by the time it runs there is nothing to distinguish the two.

This route needs `h5py`, which is not part of the base install. Without it the file dialog still offers HDF5 files and opening one reports that the package is missing, which is the honest failure: nothing else in the program is affected.

Files that declare their own structure

Some exporters state what is in the file, and those are read as stated.

```
/DOMAIN/OSMA_1/PARAMETERS/ALFA                a channel, with @unit
/DOMAIN/OSMA_1/PARAMETERS/ALFA/ALFA_1_2_0     its samples, for one run
/DOMAIN/OSMA_1/SIMULATIONS/SIMULATION_1_2_0   a run, with its sweep values
```

`PARAMETERS` lists the channels and `SIMULATIONS` lists the runs, so a dataset is the two put together: nothing is inferred and nothing is ranked, and the answer cannot come out the wrong way round. Units are taken from each channel's own group, where the exporter declares them once, rather than from every dataset.

It is also the cheapest route there is. Two listings per session, one attribute per channel, and one dataset opened to learn the sample count — no dataset name is enumerated at all, because a dataset's path is its channel's and its run's put together.

Why a file with several sessions cannot be inferred

A file holding `OSMA_1` beside `OSMA_2` repeats the same run names under each. A reading keyed on the dataset name alone therefore finds `ALFA_1_2_0` twice, correctly concludes that two datasets cannot both be the same channel of the same run, and stands down — leaving a reading that is consistent and useless: one "run" named after the top of the tree, carrying every dataset in the file as a channel of its own.
Read as declared, the session is part of the run's identity, which is what it is. The runs come out as `OSMA_1/1_2_0`, `OSMA_2/1_2_0` and so on, and the channels are the channels.

Why there is no single HDF5 reader

For a file that declares nothing, the layout has to be inferred. HDF5 is a container, not a layout. It says how to store named arrays in a tree and nothing whatever about which array is a channel and which is a run, so two exporters writing the same simulation produce trees that share only the file signature. The axis they disagree about most is where a run's identity is written:

```
/RUN_1/ALTITUDE            a group per run, a dataset per channel
/ALTITUDE/ALTITUDE_1_2_0   a group per channel, a dataset per run, the
                           run spelled into the dataset name
/CASE_2/aero/ALTITUDE      the run further up the tree
/ALTITUDE                  a single run, nothing to disambiguate
```

Only one-dimensional numeric datasets are candidates, and only those of the length most of them share, so a short metadata vector sitting beside the channels is passed over rather than becoming a channel two samples long.

How the layout is chosen

Every convention above is tried. Those that do not partition the file consistently — two datasets claiming to be the same channel of the same run, or a run missing a channel another run has — are discarded outright. What survives is ranked on evidence taken from the tree itself:

- whether each channel's key is the name of the group its dataset sits in. `/ALFA/ALFA_1_2_0` read with the run in the leaf gives the channel `ALFA` inside the group `ALFA`; read as a single run it gives a channel called `ALFA_1_2_0` inside a group called `ALFA`, which no exporter writes on purpose.

- whether the run keys are the same beneath every group. A run identity is orthogonal to the tree: each channel has one dataset per run. A key appearing beneath only one group is part of that group's name.
- having found more than one run at all, as a tie-break only, since a file really can hold one.

T

he chosen convention is written to the log at INFO level, together with the runners-up and their scores. If a file ever opens with its runs and channels the wrong way round, that line is the first place to look.

Units come from a `units` or `unit` attribute on each dataset where one exists, and are left empty otherwise — the same position as the packed `.aoe` dialect, where the file declares none.

What is not read

Attributes carrying anything other than a unit — masses, inertias, configuration strings — are ignored. They are not channels, and there is no place in the interface where a per-run property belongs today.

How it reaches the core

The core has no HDF5 parser and gains no new compiled dependency. The file is read in Python and its columns handed down through four calls on the C ABI (`aoe_begin_arrays`, `aoe_add_arrays_sim`, `aoe_set_arrays_column`, `aoe_end_arrays`); what the core ends up holding is the same header table, run list and per-run columns that parsing an `.aoe` file would have produced.

Opening the file reads *no samples at all*. The names, the runs and the shapes are enough to build the DATA tree and the parameter lists, and a column is fetched the first time something asks to draw it — a page of four tiles against forty channels reads four of them. A column can arrive from three places, tried in order of cost: already in memory, from the sidecar, or read from the file. An ISO scan is cheaper still, reading element zero of each run's dataset rather than the column.

A moment after the file is on screen, and only above 8 MB, a separate process fetches the rest and writes the `.aoec` sidecar (§7.4.1). Every later open is then served from it, and the HDF5 file is not read at all.

Reading one over a network

A share is slow in round trips rather than in bytes, and working out the layout is where the round trips used to be. Three things keep the count down.

An inferred layout is worked out from paths, not from datasets. None of the conventions above looks at a dataset's contents — every one keys on the path — so the names are enough to decide which is in force. Enumerating names is one traversal; opening every dataset to ask its shape is a round trip apiece, and a file of 50 runs by 1000 channels holds fifty thousand of them. Shapes are read for the channels of the first run only, which settles the common length and collects the units in the same pass, plus one dataset of each other run to confirm it agrees: **1,049 reads rather than 50,000**.

The layout is written down. It is the same answer every time, so it is stored, gzipped, as a `.aoelay` file beside the source — or in the per-user cache directory when the source's own is read-only, which solver output usually is, under the same `AOE_CACHE_DIR` override the sidecars use. Opening the file again reads that instead of walking anything: **one read rather than 50,000**.

Three things have to hold before it is believed: the source must be the size and date it was when the layout was written, the document must be a format this version understands, and one dataset the layout names must still be there with the length it claims. The first two are cheap and not proof; the third is a single read and is what makes them safe to trust. Anything that fails sends the reader back to working it out properly.

The file is opened once per task. A handle is kept and reused, so drawing six parameters, or scanning fifty of them for an ISO filter, costs one open rather than six or fifty. It is released when the task finishes — reads only happen inside a task, so that is the whole span over which reuse pays, and holding a file open while nothing is reading it would risk being the reason a solver's own write is refused. Before a handle is reused the source is stat-ed, because a handle reads the file it was opened on, which is not necessarily the file at that path any more.

Why this matters most once a sidecar exists

At that point not one sample is read from the HDF5 file. The tree was still being walked on every open, though, because the core has to be told what the file contains before it can look for a sidecar — so the walk was the entire cost of opening an already-cached file, and all of it was waste.

A sidecar for a pushed file needs every column resident first. The core writes one from what it holds and has no reader of its own to fill a gap, so the process that writes it fetches the whole file before writing anything. On a file larger than the machine's memory that fails and no sidecar is written; the file is still perfectly usable, column by column.

Reload (DATA panel) works and goes back through the reader rather than through the core, keeping the file's index and with it every style keyed on its runs.

Chapter 9

How the data is read

9.1 Memory mapping

The file is memory-mapped, read-only, in one view covering the whole file. There is no chunked or windowed mapping and no explicit page handling; demand paging is left entirely to the operating system, which is the right choice for a file that will be read column-by-column in an order the program cannot predict.

On POSIX the file is opened with `O_RDONLY | O_CLOEXEC`, sized with `fstat` — a size of zero or less is a failure — and mapped with `PROT_READ` and `MAP_PRIVATE`, with `madvise` asking for sequential access ahead of the index scan. On Windows the path is explicitly widened from UTF-8 to UTF-16 before `CreateFileW`, because the ANSI entry points failed silently on non-ASCII paths, and the share mode permits a solver to keep writing the file while it is open. There, asking for sequential access is a deliberate no-op: the flag would have to be set at open time and it measurably hurts the random column access that follows.

The modification time is read per platform — nanoseconds on POSIX, 100-nanosecond ticks on Windows — and returned as an opaque, monotonically increasing integer whose units differ between platforms and must never be interpreted. It is only ever compared for equality.

9.2 One index pass, then lazy parallel columns

Opening a file performs a single linear walk that records only the header names, the units, how many values appeared on the first data line of a run, and for each run its identifier, its unique key and the byte offsets at which its data begins and ends. No sample data is read. Opening a 578 MB file is therefore a scan, not a parse, and if a valid sidecar attaches the scan does not happen at all — headers, units, run identifiers, byte ranges and the values-per-line figure all come straight out of the sidecar's header.

Samples arrive later, per column and per run, the first time something asks for them. Each load maps the file afresh, clamps every recorded byte range to the file's current length — the solver this tool watches truncates and rewrites its output, so the index may have been built against a longer file — computes which columns are not already resident, divides the work into units, and dispatches them across the thread pool. Nothing already cached is re-read. Once parsed, the per-column chunks are stitched together outside the per-run locks, summarised in parallel, and published.

A byte range is split for parallelism only when the file is known to be one record per line, and only when the span exceeds 2 MiB; chunk boundaries are then advanced to the next line feed so that no line is ever split across two workers. The summary pass and the sidecar's copy path both run across the pool as well.

No SIMD, no OpenMP

It is worth saying explicitly, because it is often assumed of code this fast: there are no vector intrinsics and no OpenMP anywhere in the core. A search across the sources and build files for `immintrin`, `__m128`, `__m256`, `-mavx`, `-msse`, `-march=` and `-fopenmp` returns nothing. The parallelism is an ordinary thread pool; the only vector-like trick is scalar

SWAR in the token skipper.

9.3 The inner loop

The innermost loop of the program is the one that turns bytes into floats, and it is written accordingly. A flat vector maps column index to output slot, because a hash lookup per token was measurably too slow. Output vectors are pre-reserved from the observed bytes-per-line of the first line rather than grown. The column counter is declared outside the line loop, which is what carries a wrapped record across newlines. Wanted columns go through the float parser; unwanted columns go through a much cheaper token skipper. Slack is returned to the allocator only when there is enough of it to be worth a reallocation.

Text-to-number conversion is hand-rolled rather than delegated to `strtod`, `strtof` or `std::from_chars`. The reasons are recorded in the header: `strtof` is correctly rounded but locale-aware and roughly an order of magnitude slower, and `std::from_chars` for floating-point types is missing from libstdc++ before version 11 and from older libc++. (`strtod` is used, but only for numeric literals in the formula parser, where performance is irrelevant.)

The token skipper uses the classic SWAR has-zero idiom over a 64-bit word loaded by `memcpy` rather than by a cast — a `reinterpret_cast` there is undefined behaviour that some compilers do miscompile at `-O3` — and locates the first separator byte with a count-trailing-zeros intrinsic. Big-endian builds fall back to a plain byte loop. The measured effect on a 500 MB file is about 2 GB/s against about 1.1 GB/s for a byte-at-a-time skip.

9.4 What happens to malformed input

The core throws no exceptions for malformed data and logs nothing. Every failure mode has a defined, quiet behaviour.

Situation	Behaviour
Non-numeric token	becomes a quiet NaN, token consumed; never zero
The float parser cannot advance	the rest of the line is abandoned
A line that does not start with a digit, sign or point	skipped; the column counter is not reset
A blank line inside a run	skipped
Wrong number of values on a wrapped record	the column counter drifts, and columns silently misalign from there on
The file shrank since it was indexed	byte ranges are clamped to the current length
The file cannot be mapped at load time	columns are marked loaded with an empty block, so the interface does not retry every frame and the channel reads as absent rather than as a line of zeros
Empty or zero-length file	mapping fails, so opening fails
No `*PARAMETRES` block	opening fails
More units than headers	the unit list is resized to the header count
The sidecar disagrees with the dataset's shape	the sidecar is ignored and the text parse takes over

9.5 The `.aoec` sidecar

The first time a file of 8 MB or more is opened, the same background task that opened it also writes a sidecar beside it. Thereafter opening that file skips the index scan and loading a column becomes a copy out of a mapping.

9.5.1 Layout

The file begins with a fixed 128-byte header, followed by one 16-byte record per column, one 24-byte record per run, then the sample data aligned to a 64-byte cache line. The header carries the magic `"AOECACHE"` — eight bytes, not NUL-terminated — a format version of 1, a byte-order mark of `0x01020304`, the column and run counts, the values-per-line figure, and the source fingerprint. What is stored is column-major `float32` sample arrays, plus the complete index — headers, units, run identifiers, run byte ranges in the source, values per line — plus per-parameter-per-run summary statistics: minimum, maximum, whether any value is finite, whether there are gaps, and whether the channel is monotonic. No text is stored, no decimated form, and no formula results.

9.5.2 Deciding whether it is still valid

Three independent properties are compared for exact equality, and all three must match.

Field	Catches	Would miss on its own
size	truncation, extension, append	an in-place rewrite of the same length
modification time	an in-place rewrite	anything that restores timestamps: `rsync --times`, a restore from backup, some build systems
content hash	a rewrite that preserved both of the above	nothing — it is the backstop

The hash is 64-bit FNV-1a, chosen explicitly for being verifiable at a glance: it guards against a file changing underneath the program, not against an adversary crafting a collision. It is seeded with the file's length, so a file that is a strict prefix of another cannot collide with it, and the length is taken from the same open handle that the bytes are read through, so length and content always describe the same file. Files of 4 MiB or less are hashed in full. Larger files are sampled: sixty-four evenly spaced windows of 64 KiB, the first at offset zero and the last ending at the end of the file, with each window's offset folded into the hash before its bytes so that identical blocks moved to a different place in a rearranged file do not cancel out. The cost of validating a sidecar is therefore capped at about 4 MiB of reading however large the source is — which is exactly why the advantage of a sidecar widens as files grow.

9.5.3 Where sidecars live

Two locations are tried, nearest first. Beside the source, the sidecar is the full source name with `.aoec` appended — `run.aoe` becomes `run.aoe.aoec`, keeping the original extension so it can never collide with a differently-suffixed file of the same stem. Failing that, a per-user cache directory is used, with the full source path hashed into the filename so that two files called `results.aoe` in different directories do not share a cache.

Platform	Per-user cache directory
Windows	`%LOCALAPPDATA%\visimu\cache`, falling back to `%APPDATA%`, then `%TEMP%`
macOS	`$HOME/Library/Caches/visimu`
Linux and others	`$XDG_CACHE_HOME/visimu`, else `$HOME/.cache/visimu`

`AOE_CACHE_DIR` overrides that fallback location entirely. Each candidate is tried in turn and the first that both opens and matches is used; one that fails to match is skipped rather than treated as fatal, because the other location may hold a current one. Caching can be switched off entirely by setting `AOE_CACHE` to `0`, `false`, `no` or `off`; the variable is read once per process, because an environment lookup on Windows is a system call.

9.5.4 What it buys

On a 73 MB generated file of 1200 runs by 1000 rows by 6 channels, measured end to end:

Operation	From text	From sidecar
Open (index)	213 ms	22 ms
Preload 2 channels across 1200 runs	81 ms	7 ms
Fetch 1200 curves	6 ms	5 ms
ISO scan	7 ms	0.3 ms
Total	**306 ms**	**32 ms**

Roughly a tenfold improvement. Writing the sidecar took 0.24 s once, and it occupies 27.7 MB against a 73.1 MB source — `float32` is more compact than the decimal text it came from. The two structural wins are that opening does not scan, and that loading a column becomes a copy out of a mapping plus a copy of the precomputed summary, so the summarising pass does not run at all.

The 8 MB threshold lives in the interface, not the core

`MainWindow.CACHE_THRESHOLD_BYTES` is `8 * 1024 * 1024`, and the decision to write a sidecar is made there. Below that size, parsing is quick enough that a sidecar would save nothing measurable and would only leave an extra file beside the results. The consequence is that a script using the Python API gets no sidecar unless it calls `build_cache()` itself. Every reason to skip is silent: a read-only share, a full disk, caching switched off.

9.6 The memory budget

The core owns all sample memory and reports how much it is holding. When that figure exceeds the configured limit — 2048 MB by default, floored at 64 MB — the interface asks the core to trim to 80 % of the cap, which gives the eviction some hysteresis instead of trimming on every redraw. Eviction is least-recently-used by column, and is always safe: an evicted column is simply re-read the next time something asks for it.

Eviction can never free a buffer a visible curve is reading from. Arrays handed to Python are read-only views of the core's memory rather than copies, and each carries a pin object as its NumPy base which keeps the underlying block alive for as long as any view exists. That is also why an eviction invalidates the fast redraw path: what was resident before says nothing about what is resident now, so the next redraw re-checks.

Chapter 10

Computed parameters: the formula language

A computed parameter is a named expression over the channels that exist, evaluated in the core, per run, and thereafter indistinguishable from a channel the file carried. This chapter is the complete language reference.

10.1 How an expression is executed

An expression is compiled once into a flat bytecode and evaluated by a stack machine vectorised over rows. It is deliberately not an abstract syntax tree walked per sample. Compilation happens once per formula rather than once per run; the bytecode stores variable slots and data is bound at evaluation time, so no pointer can dangle when a later column load reallocates a vector. There is no per-node allocation and no virtual call.

The evaluator tracks whether each stack entry is a broadcast scalar or a real buffer, so `2 * 9.81` collapses to a single float rather than filling a million-element array, and buffers are drawn from a reused pool so that a deep expression allocates a handful of buffers rather than one per operator.

10.2 Operators

Level	Operators	Associativity	Meaning
1 (lowest)	`||`	left	logical or
2	`&&`	left	logical and
3	`== !=`	left	equality
4	`<= >= < >`	left	comparison
5	`+ -`	left	addition
6	`* / %`	left	multiplication, division, remainder
7	unary `- + !`	right	sign and logical not
8 (highest)	`^`	right	power

Because `^` is right-associative, `2^3^2` is 512 rather than 64.

Three behaviours are worth knowing before they surprise you. Division by zero yields not-a-number, never an infinity: the reason is stated twice in the source, that an infinity silently destroys the plot's y range whereas a NaN simply breaks the line where the data is bad. The same applies to `%` and to `mod`. Comparisons yield exactly 1.0 or 0.0, so they can be multiplied into arithmetic. And the logical operators do not short-circuit — both sides are always evaluated — which matters only in that a costly right-hand side is not skipped.

The power operator binds tighter than unary minus on its left, so `-2^2` is $-(2^2)$, matching engineering calculators, while `2^-3` still works because the exponent side is parsed as a unary expression. Logical not never swallows an inequality, and `!NaN` is `0.0`.

10.3 Element-wise functions

These operate one sample at a time.

Function	Behaviour
`abs(v)`	absolute value
`sqrt(v)`	NaN for a negative argument
`sin cos tan`	as the C library
`asin(v) acos(v)`	NaN outside $[-1, 1]$
`atan(v)`	arctangent
`exp(v)`	exponential
`log(v) ln(v)`	natural logarithm; both names map to the same operation, and a non-positive argument gives NaN
`log10(v)`	base-10 logarithm; non-positive gives NaN
`floor ceil round`	as the C library
`sign(v)`	-1, 0 or $+1$; NaN gives 0
`deg(v)`	radians to degrees
`rad(v)`	degrees to radians
`min(a,b) max(a,b)`	per-sample smaller or larger; NaN if either is NaN
`atan2(a,b)`	two-argument arctangent
`pow(a,b)`	as `a^b`
`mod(a,b)`	remainder; NaN when `b` is zero
`hypot(a,b)`	$\sqrt{a^2 + b^2}$ without intermediate overflow
`clamp(x, lo, hi)`	constrains `x`; NaN stays NaN
`if(cond, a, b)`	`a` when `cond` is non-zero and not NaN, else `b`. A NaN condition takes the false branch, and both branches are always evaluated

10.3.1 The three logarithms

Which logarithm a name gives is the easiest thing in this language to assume wrongly, since a pocket calculator's `log` is base ten and this one is not. There are two functions under three names.

Name	What it computes
`log(x)`	The natural logarithm, base $e = 2.71828\ldots$ — *not* base ten. This follows C and Fortran rather than a calculator keypad.
`ln(x)`	Exactly the same function as `log`, under the name most engineering texts use. Write whichever reads better; the two give identical results.
`log10(x)`	The base-ten logarithm. This is the one for decibels, orders of magnitude, and anything you would read off a logarithmic axis.

All three return NaN for an argument that is zero or negative rather than an infinity, for the reason given above: a NaN breaks the line where the data is out of domain, an infinity would destroy the plot's y range. The inverse of `log` and `ln` is `exp(x)`; the inverse of `log10` is `pow(10, x)`.

```
DECIBELS = 20 * log10(AMPLITUDE / AMP_REF)
DECADES  = log10(ALTITUDE)
TAU      = -1 / deriv(ln(ENERGY), TIME)
```

Taking a logarithm is not the same as drawing on one

The *Log Y* checkbox in STYLE draws an existing channel on a base-ten axis and leaves the data untouched. Use that to look at a channel logarithmically. Use these functions when you want the logarithm itself as a channel — to take a slope off it, say.

10.4 Whole-series functions

These cannot be evaluated one sample at a time — a derivative needs its neighbours, a filter needs its own previous output — so they operate on the finished buffer.

Function	Behaviour
`deriv(Y, X)`	dY/dX by central differences, one-sided at the two ends, accumulated in double precision. A zero step gives NaN rather than an infinity. A single sample gives zero
`integ(Y, X)`	cumulative trapezoidal integral starting at zero. NaN steps contribute nothing but the running total is still written out
`lowpass(Y, X, fc)`	first-order recursive low pass with the cutoff in hertz. The timestep is taken from `X` at every sample, so uneven sampling is handled correctly. A non-positive cutoff performs no filtering at all rather than dividing by zero
`highpass(Y, X, fc)`	the matching first-order high pass; the first output sample is zero
`movavg(Y, N)`	centred moving average over `N` samples, computed by prefix sum so the cost is independent of the window size. NaN samples are excluded from both the sum and the count; an all-NaN window gives NaN
`delta(Y)`	`Y` minus its first sample, so a leading NaN makes the whole channel NaN
`min(Y) max(Y) mean(Y)`	reductions to one number, accumulated in double and ignoring NaN and infinity, then held flat across every sample so the result combines with per-sample channels by ordinary arithmetic

Overload resolution is by argument count, not by name. `max(NZ)` is the run's peak; `max(NZ, NY)` is the per-sample larger of two channels. Each run is reduced separately, which is what makes `NZ / max(NZ)` normalise every run against its own peak. If a name exists but no definition has that arity, the error names the counts that would have worked — *min() takes 1 or 2 arguments, not 3*.

Passing a scalar where a series function expects its `X` argument fills the result with NaN, because a constant x would make every step zero.

10.5 Constants and the lookup form

Three constants are recognised, case-insensitively: `pi`, `e` and `nan`. There is no `inf`, and no `true` or `false` — comparisons already yield one and zero.

The lookup form reads one channel at the moment another channel satisfies a condition, and broadcasts that single value across the whole series:

```
NAME[COND=value]    # NAME at the sample where COND is closest to value
NAME[COND=first]    # NAME at the first sample where both are real
NAME[COND=last]     # NAME at the last sample where both are real
```

For a numeric target the sample minimising the absolute difference wins; samples whose condition is NaN are skipped, though a NaN value at the winning condition is returned as it stands. For `first` and `last` the scan runs from the respective end looking for the first index at which both the condition and the value are real numbers, so leading and trailing gaps are skipped and the lookup returns an actual sample rather than NaN. If nothing is found the result is NaN.

10.6 Worked examples

```
LOAD_FACTOR  = NZ * 9.80665
PITCH_RATE   = deriv(THETA, TIME)
DISTANCE     = integ(VTAS, TIME)
CLIMB_RATE   = deriv(ALTITUDE, TIME)
ALPHA_SMOOTH = lowpass(ALPHA, TIME, 2.0)
ALPHA_NOISE  = highpass(ALPHA, TIME, 5.0)
DRIFT        = HEADING - HEADING[TIME=first]
ALT_GAINED   = ALTITUDE[TIME=last] - ALTITUDE[TIME=first]
HIGH_MACH    = if(MACH > 0.8, ALPHA, nan)
ENERGY       = 0.5 * MASS * (VX^2 + VY^2)
NZ_NORM      = NZ / max(NZ)
NZ_ENVELOPE  = max(NZ, NY)
ALPHA_EXCESS = ALPHA - mean(ALPHA)
```

`HIGH_MACH` is worth a second look: substituting NaN for the samples you do not want breaks the line there, which is how you isolate a flight phase without editing the data.

10.7 Names, dependencies and errors

Any identifier that is not a call, a lookup or one of the three constants becomes a variable. Names are resolved after alias resolution, and a name must be either a physical column of some open file or another computed channel — otherwise the definition is refused with *unknown parameter 'X'*.

Formulas may reference other computed parameters, and the dependency graph is checked at definition time rather than discovered at plot time. The new definition is installed tentatively, the graph is walked with a depth limit of 32, and on detection the previous definition is restored and the error reads *'NAME' would depend on itself*. Exceeding the depth limit is reported the same way. Removing a parameter removes everything defined in terms of it, and the interface reports which.

Every variable is bound once, up front; a variable that resolves to nothing is a hard failure reading *no data for parameter 'X'*. The sample count is the shortest of the bound channels, and is never allowed to exceed it — the engine never reads past the shortest channel involved. A formula with no variables at all, such as `pi` or `2 + 3 * 4`, would otherwise have no length to fill, so the run's first column is consulted for a row count, falling back to a single row.

Validation, which is what runs as you type in the COMPUTE tab, compiles the expression and checks every variable against the set of known names without touching any data. The distinct compile-time messages are *empty expression*, *unexpected trailing text*, *expected ')'*, *malformed number*, *unexpected character*, *expected ',' or ')' in function arguments*, *unknown function 'X'*, the arity message, four specific lookup messages, and *invalid expression* as a last resort. The lookup messages are unusually helpful and worth quoting:

```
expected a parameter name inside [ ]
expected '=' in lookup, e.g. SPEED[TIME=0.5]
```

```
expected a number, 'first' or 'last' in lookup,
    e.g. SPEED[TIME=0.5] or SPEED[TIME=last]
expected ']' in lookup
```

Two consequences of caching

Results are cached per run and per name, and failures are cached too, as an empty block. Without that the interface would retry the same doomed evaluation on every repaint; and the tempting alternative — substituting a zero — would turn a channel that could not be computed into a flat line that looks like data.
A computed channel also carries no unit. Deriving one through a formula would mean dimensional analysis, and a wrong unit on an axis is worse than none.

Chapter 11

The algorithms behind the plots

11.1 Decimation: drawing what the screen can show

A tile with twenty-five runs of half a million samples cannot hand two and a half million points to a plotting library and stay interactive. Nor can it keep every thousandth sample: stride sampling silently deletes every spike between the samples it keeps, and on flight-test data that is exactly the part an engineer is looking for. visimu therefore uses min/max bucketing.

The visible index span is divided into exactly `max_points` buckets by index, so bucket sizes differ by at most one sample. Within each bucket the minimum and maximum y are found, skipping NaN, and both are emitted in x order — swapped if necessary so the polyline never doubles back. Where a bucket's extremes coincide, one point is emitted; where a bucket is entirely NaN, a single NaN is emitted at the bucket's first x, so the line breaks visibly instead of interpolating across missing data. The output is therefore at most $2 \times$ `max_points` points. Below a count of twice the budget there is nothing to gain, and the span is copied through verbatim.

What the guarantee is, and is not

Every bucket's extremes survive, so the drawn envelope is pixel-accurate: no peak or trough can be lost at any zoom level, because it is either the minimum or the maximum of whatever bucket it falls into. What is not preserved is the shape between extremes within a bucket, nor the exact ordering of intermediate samples. NaN gaps are preserved as gaps, at bucket granularity.

11.1.1 Selecting the visible window

Before bucketing, the visible x window must be turned into an index span, and this is where a subtlety lives. When x is known to be non-decreasing, two binary searches find the span in logarithmic time. When it is not — and a phase-plane plot such as Mach against altitude over a climb and descent is not — a full scan finds the outermost indices whose x falls inside the window and keeps everything between them.

The reason is a real defect the scan fixes. On a climb-and-descend trajectory x rises and then falls, so a binary search brackets only the first crossing and the entire descending half of the trajectory silently disappears — differently depending on where you have panned to. One sample is also kept on each side of the window, so the curve is drawn to the plot border rather than stopping just short of it.

11.1.2 The pixel budget

The front end asks for roughly two points per horizontal pixel — one for each bucket's high and low — so a 600-pixel tile draws about 1200 points whether the channel holds ten thousand samples or ten million. A second ceiling of 1 500 000 points applies across all the curves of one tile together, so that a page with many curves cannot overwhelm Qt.

That second ceiling is a named function rather than an inline expression for a reason recorded in the source: written as a bare division it acted as a floor at moderate curve counts,

raising the target from 1800 to 10 000 at seventy-five curves and pushing one and a half million points into Qt on every pan.

The visible window is padded by five per cent on each side so the curve stays drawn to the tile's edges while panning. The quoted end-to-end effect on a heavy page is a sixty-fold reduction: 15.9 million samples become 266 thousand drawn points across 225 curves.

11.1.3 Statistics over the same window

The statistics shown in INSPECT use Welford's one-pass method rather than accumulating a sum of squares, because a sum of squares loses precision on a channel with a large offset — altitude in feet, squared, overflows single precision quickly. The reported standard deviation is the sample standard deviation and is zero for a single sample. NaN and infinite samples are skipped. On a non-monotonic x with a window given, every sample is tested individually rather than bracketing a span, for exactly the phase-plane reason above.

11.2 The redraw cache

Most rebuilds do not change a single drawn point. Toggling the grid, restyling one colour, dragging a marker, editing the legend, coming back to a page — all of them arrive at `rebuild()`, and the naive answer is to throw away every item and re-decimate every curve from full resolution to arrive at a picture identical to the one already on screen. Four caches stand between a rebuild and that work.

11.2.1 The resample key

The decimation result for a group of curves is tagged with a resample key: the quantised lower edge of the view, the span rounded to twelve significant figures, the point target, and whether the axis is logarithmic. A group whose key is unchanged is skipped entirely.

Both halves of that key are necessary. Quantising only the endpoints hides every zoom whose lower edge scales with its width — the windows $0 \ldots 1$ and $0 \ldots 0.25$ quantise into the same bucket — so zooming in around the origin gained no resolution. Rounding to twelve significant figures is needed because a view range read back from a plot wobbles in the last bits of a double, which would otherwise invalidate the cache on every mouse move.

11.2.2 Adopting the drawn items

Each group records a signature over its curves: the curve's identity, and the address and length of the x and y buffers behind it. A reloaded file, or a column that was evicted by the memory cap and re-read, lands at a different address and so compares unequal. When the signature matches, the existing graphics item is adopted and merely restyled; the group keeps its resample key, so no points are recomputed either.

Comparing addresses is sound only because the previous groups are still alive and still referencing those buffers while the comparison runs — a buffer that is still referenced cannot have been freed and another put at its address. That is why `rebuild()` holds the old groups rather than dropping them first.

One item per pen

Curves that share a pen cannot be told apart on screen, so they are concatenated into a single polyline with one NaN between consecutive runs and drawn with a connect rule that breaks the line there. Twelve hundred runs times two parameters — 2400 graphics items — becomes about twelve. Everything except painting is still per curve: the crosshair

readout, click-to-inspect, marker dots, area segments and the CSV export all need to know which run and which parameter a sample belongs to.

11.2.3 The plot's own settings

Grid, log mode, axis captions and the title are written only when one of them differs from what was last written.

They were re-applied on every rebuild, and most rebuilds change none of them. `setLogMode` is what made that expensive rather than merely redundant: pyqtgraph propagates it to every data item on the plot, which invalidates each curve's cached display data, and then re-enables auto-range — which the tile then has to undo in its own view restoration. A rebuild the two caches above had just arranged to cost nothing threw away the transformed points of every curve on the tile and moved the frame, twice, for a setting nobody had touched.

The title is guarded in the same comparison rather than separately: it is a pure function of the tile's parameters and the theme's foreground colour, so one comparison answers for all of it. A placeholder message writes over the title, so showing one forgets the comparison — without that, a tile that showed *loading…* while a file opened kept the placeholder as its heading.

11.2.4 The drawn annotations

Markers and shaded bands keep their items unless the annotations, the tile's x channel, whether they can be dragged, or the curves have changed.

This is the same idea applied to the overlay pass, and it matters for the same reason the item adoption does. The dot form of a marker searches each curve for the sample where the condition holds, and the band form redraws a stretch of every curve it crosses, so a thousand-run tile paid for both on every rebuild — including the rebuilds that changed neither a curve nor an annotation.

The curves are part of that comparison because the dot sits *on* a curve and the band *is* a stretch of one; they are compared by buffer address, with the same soundness argument as above.

One ordering detail is load-bearing. The overlay list is shared with the 3D data box and its tick values, which the 3D rebuild draws before the overlay pass runs. So the pass returns early on a 3D tile *before* clearing anything, and the 3D rebuild owns the clear on the way in — which is also what takes away two-dimensional markers that have no meaning in a projection.

11.3 ISO grouping

Automatic grouping divides the observed range of the distinct values into evenly spaced intervals. The extremes are found ignoring NaN and infinity; the step is computed in double precision; and the last interval's upper bound is set to the observed maximum exactly, rather than recomputed, so accumulated floating-point error cannot leave the largest sample outside every group. A sweep with a single distinct value yields one group whose bounds are equal, rather than several degenerate empty ones.

11.3.1 Boundaries

Every interval except the last is half-open, $[\min, \max)$; the last is fully closed, $[\min, \max]$. A value that is not a number belongs to no group. Two clamping fallbacks catch a value sitting a hair outside the extreme bounds through rounding — without them the maximum sample of a

sweep can vanish — while a value between two non-adjacent manual intervals still belongs to none.

A documented inconsistency

The core header describes group bounds as inclusive at both ends, and the Python-side helpers implement inclusive-at-both-ends with the first claiming group winning. The core's own `iso_group_index` is half-open except for the last interval. The two agree on every value except one sitting exactly on a shared edge, which the core assigns to the later interval and the interface to the earlier one. In practice this is invisible — the edge is a value no sample holds, because the edges are derived from the observed range rather than from the samples — but it is a genuine discrepancy.

Where grouping is configured but the interval list is empty, nothing is filtered. Nothing sorts the interval list after the user has edited it, so overlapping manual bounds are resolved by the first match in the order given.

When a new file is opened, an automatically grouped parameter is rebuilt over the new range and the active flags are carried across by position, so deselections survive. A manually grouped parameter keeps its bounds, and only names that still look automatically generated — empty, or containing the exact separator `" – "` — are refreshed, so hand-typed labels are preserved.

11.3.2 How values are written

ISO values are formatted to two decimals with trailing zeros trimmed, so `0.750000` becomes `0.75` and `3.000000` becomes `3`. Values that round to zero, and values of a million or more, are written as a power of ten with a compacted exponent: `1.20e+06` becomes `1.2e6`. A negative zero is normalised to `0`.

The small-end boundary is decided by rounding rather than by a threshold, and the reasoning is worth knowing because it affects the labels you see. A value arrives as a single-precision float, so the literal `0.01` is stored as `0.00999999978`; comparing that against 10^{-2} would send `0.01` itself to scientific notation — the one value a Mach or tolerance sweep is most likely to contain. Asking instead whether the rounded form is still non-zero puts the boundary where it belongs. The large end genuinely is a threshold, and it is about label length: past a million the integer part alone is longer than the value written as a power of ten.

11.4 Legend order, and why it stopped shuffling

A legend row is ranked by an intrinsic property of what it describes, not by the order the curves happened to be walked in. In ISO mode a grouped label ranks by its position in the grouping and an ungrouped label by the value itself; the lowest rank wins when two runs share a label. Groups are then sorted by a tuple: ISO rank, parameter order, file, run when colouring by ISO; parameter order, style-ISO rank, file, run when colouring by parameter; file, run, style-ISO rank, parameter order when colouring by simulation.

Before this, the order was the order in which each group was first seen while walking the selection, so unticking a single run could reshuffle 0–10, 10–15, 15–20 into 15–20, 0–10, 10–15. Nothing about the data had changed; only the walk had.

The style-ISO rank joined the tuple in 2.4.0, with the legend text that made it necessary: a tile coloured by parameter and dashed by ISO now emits one row per ISO value (§4.6.5), and without an intrinsic rank for those rows they came out in the order the runs were walked — the same reshuffling this section exists to have ranked away.

11.5 Logarithmic axes

pyqtgraph's logarithmic mode does not scale the axis: it plots the base-10 logarithm of the samples on a linear axis and relabels the ticks. Everything crossing the view boundary is therefore in log units, while a tile's stored minimum and maximum are, and must remain, in data units. A dedicated module owns that conversion, and it exists because of a specific defect: ticking *Log Y* on a channel spanning 1 to 1000 framed the view at log-space 1 to 1000 — data 10^1 to 10^{1000} — while the curve sat at log values 0 to 3. The tile looked empty, double-clicking recomputed the same answer, and those mixed numbers then propagated as a variable-wide limit onto tiles that were not in logarithmic mode at all.

Converting a value to view space returns nothing at all when the value is non-positive, so the caller must decide what to do rather than being handed a silently wrong number. When the low end of a requested range has no logarithm, the frame is taken as three decades below the high end rather than refusing to frame at all. At draw time non-positive samples are blanked to NaN so the drawable part of the curve still shows.

11.6 The three-dimensional projection

The projection is orthographic, and deliberately so: these plots exist to have values read off them, and under perspective the same interval covers different distances depending on where it sits in the frame.

Each of the three channels is normalised independently onto $[-0.5, 0.5]$, so channels with unrelated units stay readable together; a degenerate or non-finite range collapses to zero, which puts a channel that never moves in the middle of the cube. The path is then rotated about the vertical axis by the azimuth, tilted by the elevation, and depth is dropped:

$$r_x = x\cos A - y\sin A, \qquad r_y = x\sin A + y\cos A,$$
$$\text{screen}_x = r_x, \qquad \text{screen}_y = z\cos E - r_y\sin E,$$

with depth $z\sin E + r_y\cos E$ retained only for ordering what is drawn in front of what.

A projected path cannot be min/max decimated, because that needs a monotonic x and a trajectory through space is monotonic in none of the three axes. Plain striding is the honest option here, and it is what is used, against a budget derived from the tile width as in the two-dimensional case.

Rotating does not rebuild the tile. Only the grid, the triad and the projection are redone; the channel reads, the bounds and the thinning are left alone, which is what makes dragging to rotate smooth rather than a sequence of full reloads.

The back-wall grid picks, for each axis, the face lying behind the data from the current viewpoint, so the grid never draws over the trajectory and swaps faces as you turn it. Its tick values are chosen in steps of 1, 2, 2.5 or 5 times a power of ten, counted rather than accumulated so they cannot drift outside the box, and the whole grid is drawn as a single graphics item. The orientation triad is parented to the view box rather than to the data, so panning and zooming never reach it and it keeps a constant size; its axes follow the universal convention of red, green and blue for x, y and z, and are painted back to front by depth.

Chapter 12

The C ABI

Everything above the core is written without a C++ compiler, and the reason is this interface: sixty exported functions taking only C types and opaque handles. It is declared in `core/include/aoe/aoe_c_api.h`.

`aoe_version()` returns `"2.0.0"`, which is the core's version and not the application's (2.4.0); `-version` prints both.

12.1 Handles, versioning and errors

```
typedef struct aoe_session aoe_session;   /* opaque; the API's whole context */
typedef struct aoe_pin     aoe_pin;       /* opaque; keeps a block alive */
```

A pin holds the sample block itself, not a path back to the session, so releasing one after the session has been destroyed is well defined. That matters because on the Python side pins are released by the garbage collector, in its own order and at its own time.

The only version marker is `aoe_version()`, which returns a string. There is no numeric ABI-version symbol and no version macro in the header.

Errors are reported per session. `aoe_last_error()` returns the message from the most recent failure, or the empty string. It is cleared at the start of every mutating call but not by read-only calls, so it means the last failure since the last mutating call rather than strictly the last call. Failure is signalled by a sentinel that depends on the return type: -1 for a function returning an index, 0 for a boolean or a count, an empty string — never a null pointer — for a string, a null pointer for a float array, and zero for a size. Index arguments out of range are errors rather than crashes, and a null session pointer is handled without being dereferenced.

Exceptions never cross the boundary

Every session-taking entry point funnels through a guard that catches `std::exception`, putting its message into the session's error slot, and any other exception as *unknown error*. Unwinding out of a C function is undefined behaviour, and with `ctypes` on the other side it would take the interpreter down.

12.2 Thread safety

The session carries a reader/writer lock — an `SRWLOCK` on Windows, a `pthread_rwlock_t` elsewhere. Read-only entry points take it shared, so many concurrent readers are allowed; mutating ones take it exclusively and clear the error string. The mutating set is: adding, removing, clearing and reloading files; setting a file or run display name; setting and removing aliases; setting and removing computed parameters; building and discarding a cache; and the two calls that write into the session's scratch string, `aoe_resolve` and `aoe_iso_format`, which are logically reads but take the write lock because of that scratch.

The pure-mathematics functions — decimation, the series helpers, nearest-index, and the ISO grouping arithmetic — take no session at all and are fully re-entrant on caller-owned buffers. Below the ABI, each run has its own mutex guarding its column vector and its map

of computed results, which is what lets the thread pool fill different runs concurrently; an attached cache file is immutable once open and needs no locking.

One concurrency caveat in the source

`aoe_cache_path` takes only the shared lock yet writes the session's scratch string. Two concurrent readers can therefore each invalidate the other's returned pointer. Copy returned strings immediately — which `ctypes` does automatically when the result type is declared as a C string, so the Python layer is not exposed to this.

12.3 Function reference

In the table below, *lock* is the lock the call takes; *session* in a note means memory owned by the session and valid until the next call that changes the thing it came from; *core buffer* means a zero-copy alias of a column buffer.

Function	Lock	Returns, and what to know
Library		
`aoe_version()`	—	`const char*`, a static literal, always valid. The *core*'s version.
`aoe_worker_count()`	—	`int`.
Session		
`aoe_session_create()`	—	`aoe_session*`; caller owns. `NULL` on failure.
`aoe_session_destroy(s)`	—	consumes the handle; `NULL` safe.
`aoe_last_error(s)`	—	session; valid until the next call. `""` on failure.
Files		
`aoe_add_file(s, path)`	write	`int` index. `-1` and the error set on failure.
`aoe_remove_file(s, file)`	write	invalidates every pointer from that file.
`aoe_clear_files(s)`	write	invalidates all file-derived pointers; aliases and computed definitions survive.
`aoe_file_count(s)`	read	`int`.
`aoe_file_name(s, file)`	read	session. The display name. `""` on failure.
`aoe_file_path(s, file)`	read	session. `""` on failure.
`aoe_stale_files(s, out, max)`	read	`int` count; writes $\leq$ `max` into `out`. `out=NULL` just counts. Returns the full count.
`aoe_reload_file(s, file)`	write	`1/0`; drops cached samples and computed results for that file and re-reads the index. Run keys survive it.
`aoe_set_file_name(s, file, name)`	write	display only; empty or `NULL` ignored. Run keys are not rebuilt from it.
`aoe_set_sim_name(s, file, sim, name)`	write	display only; `unique_key` untouched, so colours survive.
Runs		
`aoe_sim_count(s, file)`	read	`int`.
`aoe_sim_id(s, file, sim)`	read	session; what the file calls it.
`aoe_sim_name(s, file, sim)`	read	session; falls back to the id when the display name is empty.
`aoe_sim_key(s, file, sim)`	read	session; the identity styles and workspaces are built on.
Channels and names		
`aoe_header_count(s, file)`	read	`int`.
`aoe_header(s, file, i)`	read	session.
`aoe_header_unit(s, file, i)`	read	session; parallel to `aoe_header`; never `NULL`.

Function	Lock	Returns, and what to know
`aoe_param_unit(s, name)`	read	session; searches every open file, follows aliases; `""` for computed.
`aoe_param_count(s)`	read	`int`; union of headers, computed and aliases, sorted.
`aoe_param(s, i)`	read	session.
`aoe_set_alias(s, alias, target)`	write	no-op if either is empty or they are equal.
`aoe_remove_alias(s, alias)`	write	—
`aoe_resolve(s, name)`	write	session *scratch* — invalidated by the next scratch-writing call.
Computed parameters		
`aoe_set_computed(s, name, expr)`	write	`1/0`; clears cached computed results in every file. `0` and the error set on failure.
`aoe_remove_computed(s, name)`	write	`int` count; removes transitively, names readable through the next entry.
`aoe_removed_name(s, i)`	read	session; valid until the next `aoe_remove_computed`.
`aoe_is_computed(s, name)`	read	`1/0`; resolves aliases first.
`aoe_computed_source(s, name)`	read	session; the stored expression text.
`aoe_validate_formula(s, expr)`	read	`1/0`; parses and checks variables, evaluates nothing.
Samples		
`aoe_channel(s, file, sim, name, *count)`	read	core buffer, zero-copy. Valid only until the next eviction — anything that keeps it must pin. `NULL` on failure.
`aoe_channel_pinned(s, file, sim, name, *count, **pin)`	read	as above, and the caller owns `*pin`: the samples stay alive regardless of the memory cap until released.
`aoe_pin_release(pin)`	—	frees the pin; `NULL` safe.
`aoe_pin_sorted(pin)`	—	`1` when the samples are non-decreasing. Precomputed at parse time — free.
`aoe_pin_bounds(pin, *min, *max)`	—	precomputed extent; what autoscaling needs without rescanning a million samples per curve per replot.
`aoe_pin_count(pin)`	—	`size_t`.
`aoe_preload(s, file, sims, n, names, n)`	read	loads every physical column the names need in one parallel pass; `sims=NULL` means all.
Pure mathematics — no session, fully re-entrant		
`aoe_decimate(x, y, count, x_min, x_max, max_points, out_x, out_y)`	—	`size_t` written; caller owns all buffers, `out` needs $\geq$ 2×`max_points`. Uses the whole series when `!(x_min < x_max)`.
`aoe_decimate_ex(..., x_sorted, ...)`	—	as above plus `x_sorted`: `1` sorted, `0` not, `-1` detect. Pass `0` for phase planes.
`aoe_series_sorted(values, count)`	—	`1/0`.
`aoe_series_bounds(values, count, *min, *max)`	—	`1/0`.
`aoe_nearest_index(x, count, target)`	—	`size_t`; returns `0` for an empty series, indistinguishable from a genuine hit at index 0.
`aoe_series_stats(x, y, count, x_min, x_max, out)`	—	writes 7 doubles: min, max, mean, stddev, first, last, count. `x=NULL` or an empty window means the whole series.
`aoe_series_stats_ex(..., x_sorted, out)`	—	same `x_sorted` contract as `aoe_decimate_ex`.
Memory		
`aoe_resident_bytes(s)`	read	`size_t` of sample data cached across every open file.

Function	Lock	Returns, and what to know
`aoe_trim_memory(s, keep_bytes)`	read	frees LRU columns until at or below `keep_bytes`; returns the total afterwards. Always safe — evicted data is re-read on demand.
Sidecar		
`aoe_cache_enabled()`	—	`0` when `AOE_CACHE` switches caching off.
`aoe_cache_active(s, file)`	read	`1` when a valid sidecar is attached and serving reads.
`aoe_cache_path(s, file)`	read	session *scratch* — see the caveat above.
`aoe_build_cache(s, file)`	write	costs a full parse — call off the UI thread. Memory bounded to one parameter across every run.
`aoe_discard_cache(s, file)`	write	detaches and deletes every candidate sidecar; `1` when none remains.
ISO		
`aoe_iso_values(s, file, name, out, max)`	read	`int` count; writes $\leq$ `max`, returns the full count; NaN where absent.
`aoe_iso_distinct(s, name, out, max)`	read	ascending, NaN-free, de-duplicated across every open file; `out=NULL` to learn the count first.
`aoe_iso_auto_group(values, count, n, out_bounds, max)`	—	`out_bounds` receives 2 floats per group.
`aoe_iso_group_index(value, bounds, n)`	—	`int`; `-1` on failure.
`aoe_iso_format(s, value)`	write	session *scratch*; valid until the next call on this session.

Three of these — `aoe_stale_files`, `aoe_iso_values` and `aoe_iso_distinct` — return the full count rather than the number actually written, so a caller can size a buffer with one call and fill it with a second.

Two functions worth singling out

`aoe_channel` hands back a zero-copy pointer into a column buffer which is valid only until the next eviction; anything that intends to keep the samples must use `aoe_channel_pinned` and release the pin when done. And a pin answers two questions for free, because both were computed at parse time: whether the samples are non-decreasing — the precondition of any binary search over x — and what their extent is, which is what autoscaling needs without rescanning a million samples per curve per replot.

12.4 Building against it

The core is built as a shared library with optimisation on and no dependency beyond the C++ standard library and the platform's mapping and threading calls. The export macro resolves to `__declspec(dllexport)` on Windows and to a default-visibility attribute elsewhere when building shared, and to nothing when building statically. The artefact is `aoecore.dll`, `libaoecore.dylib` or `libaoecore.so` according to platform. Because the header declares only C types, any language with a C foreign-function interface can drive it; the Python binding in this repository is one client rather than a privileged one.

Chapter 13

The Python scripting API

Everything the interface can read, a script can read, through the same core and with the same performance. Importing the package is deliberately cheap: the exports are lazy, so importing `visimu` pulls in neither NumPy nor the core library, which is what lets `-version`, `-help` and `-selftest` work on a machine where the core has not been built.

```
from visimu import Session, FileHandle, SimulationHandle, IsoGroup, CoreError
```

13.1 The one rule

Arrays returned by `channel()` are read-only *views* of C++ memory, not copies. They are fast because of that. They carry a pin object as their NumPy base, which keeps the underlying block alive for as long as any view exists — so the memory cap can never free a buffer a script is still reading from — but they stop being meaningful when the owning file is reloaded or removed. Copy explicitly if you need to outlive that.

Asking twice for the same channel gives back the same buffer at the same address. Two caches in the plot tile are built on exactly that (§11.2), so it is a property of the API rather than an accident of it.

13.2 Session

A `Session` is created with no arguments, raises `CoreError` if the core cannot be initialised, and is a context manager. Its `last_error` property exposes the core's message.

An absent channel yields an empty array rather than an error, and callers should treat that as nothing to plot: different files in one study legitimately hold different channels. In the stats dictionary, `ranged` is true only when the x window was actually applied — a run whose x channel is shorter than its y channel cannot be windowed, and the figures are then whole-channel.

Method or property	Meaning
`add_file(path) -> FileHandle`	opens a result file; raises `CoreError` on failure
`remove_file(index)`, `clear()`	unload one file, or all of them
`file_count`, `files`, `file(index)`	the open files
`stale_files() -> List[int]`	indices of files changed on disk since they were read
`parameters -> List[str]`	every name: headers, computed channels and aliases
`plottable_parameters -> List[str]`	the same list, but empty while no file is open. Use it wherever a channel is being *chosen* on the user's behalf
`set_alias(alias, target)`, `remove_alias(alias)`, `resolve(name)`	aliases
`unit(name)`, `label(name)`	the unit, and `"ALTITUDE [ft]"` — one place decides how a channel is labelled

Method or property	Meaning
`set_computed(name, expr)`	defines a computed channel; raises `CoreError` carrying the parse message
`remove_computed(name) -> List[str]`	the name plus every dependant removed
`is_computed(name)`, `computed_source(name)`	interrogate a definition
`validate_formula(expr) -> (ok, message)`	parses and checks names, evaluates nothing
`channel(file, sim, name) -> ndarray`	read-only zero-copy view; an empty array when the channel is absent
`preload(file, sims, names)`	turns N lazy single-column loads into one batched, multi-threaded read; an empty `sims` means all
`decimate(x, y, max_points, x_range=None)`	min/max bucketing, at most 2×`max_points` points
`Session.bounds(values)`	static; extent ignoring NaN and infinity
`Session.nearest_index(x, target)`	static
`Session.stats(y, x=None, x_range=None)`	static; a dict of min, max, mean, std, first, last, count, ranged
`resident_bytes`, `trim_memory(keep_bytes)`	the cache, and LRU eviction that is always safe because columns are re-read
`cache_enabled`, `cache_active(file)`, `cache_path(file)`	sidecar state
`build_cache(file)`, `discard_cache(file)`	write or delete a sidecar; building costs a full parse
`iso_values(file, name)`	one value per run, NaN where absent
`iso_distinct(name)`	sorted distinct values across every open file
`iso_auto_group(values, n)`, `iso_group_index(value, groups)`	interval arithmetic
`format_value(value)`	the same formatting the lists show

13.3 Handles

A `FileHandle` carries its index and exposes `name`, `path`, `headers`, `units` — parallel to the headers, with empty strings where the dialect carries none, so the two can always be zipped — `simulation_count` and `simulations`. It can `rename(name)`, which changes the display name only, and `reload()`, which re-reads the file from disk and leaves every run's key as it was. The simulation list is cached on the session, because building one handle costs three ABI calls and a thousand-run file would otherwise cost three thousand on every access.

A `SimulationHandle` carries `file_index`, `index`, `sim_id` — what the file calls it — `name`, which is what the user sees, and `key`, the identity styles attach to, which never changes. It can `rename(name)` and `channel(name)`.

An `IsoGroup` is a named interval: `name`, `low`, `high`, `active`, and `contains(value)`.

13.4 A worked script

```
import numpy as np
from visimu import Session, CoreError

with Session() as session:
    handle = session.add_file("study/sweep.aoe")

    # A derived channel, checked before it is relied on.
    ok, message = session.validate_formula("deriv(THETA, TIME)")
    if not ok:
        raise SystemExit(message)
```

```
session.set_computed("PITCH_RATE", "deriv(THETA, TIME)")

# One batched, multi-threaded read for everything we are about to touch.
session.preload(handle.index, [], ["TIME", "ALPHA", "PITCH_RATE"])

# Bucket the runs by their Mach number.
machs = session.iso_distinct("MACH")
groups = session.iso_auto_group(machs, 3)

for sim in handle.simulations:
    value = float(session.iso_values(handle.index, "MACH")[sim.index])
    bucket = session.iso_group_index(value, groups)

    t = sim.channel("TIME")           # read-only view, zero copy
    q = sim.channel("PITCH_RATE")
    stats = Session.stats(q, t, (5.0, 25.0))   # over 5 s .. 25 s only
    if stats is None:
        continue
    print("{0} MACH={1} group={2} peak q={3:.4g} ranged={4}".format(
        sim.sim_id, session.format_value(value),
        groups[bucket].name if bucket >= 0 else "-",
        stats["max"], stats["ranged"]))

# What a tile would actually draw for a 900 px wide plot.
t = handle.simulations[0].channel("TIME")
a = handle.simulations[0].channel("ALPHA")
dx, dy = session.decimate(t, a, max_points=1800, x_range=(0.0, 60.0))

print(session.label("ALPHA"), dx.size, "points drawn of", t.size)
print("cache", session.resident_bytes // (1024 * 1024), "MB")
session.trim_memory(256 * 1024 * 1024)
```

Two habits are worth adopting. Call `preload` once for everything a loop will touch, rather than letting each `channel()` trigger its own load — the batched read uses the whole thread pool and one mapping. And for a file of any size, call `build_cache()` once; the graphical application does this automatically above 8 MB, but a script gets no sidecar unless it asks.

Useful extras in `_core`

Beyond the façade, `visimu._core` exposes `library_path()`, the `Pin` type with its `sorted` flag and `bounds()`, `pin_of(array)` to recover the pin behind a view, the array adapters, and the module-level `VERSION` and `WORKER_COUNT`.

Chapter 14

Performance, limits and troubleshooting

14.1 Where the time goes

Three costs dominate, and each has its own mitigation. Parsing text into floats is paid once per column per run, in parallel, and is avoided entirely on the second and subsequent opens by the sidecar. Deciding what to draw is paid on every pan and zoom, and is bounded by the pixel budget rather than by the size of the data. Creating graphics items is paid when a tile's shape changes, and is minimised by concatenating curves that share a pen into one polyline and by adopting existing items whenever the data behind them is unchanged.

A fourth cost, which is neither of those, is deciding to do work that was already done. Chapter 11 lists the four caches that stand between a rebuild and the work it would otherwise repeat, and the panels carry the same idea: each is rebuilt only when its *rows* change, and a refresh that changes only which boxes are ticked writes the boxes in place.

Limit	Value and meaning
Tiles per page	25, a 5 × 5 grid. Every page always holds 25 tile objects; shrinking a grid hides them rather than destroying them
Points per pixel	2 — one for each bucket's high and low
Points per tile	1 500 000 across all its curves, as a second ceiling
Legend rows	60, after which the remainder is summarised as *. . . and N more (legend caps at 60 rows)*
ISO value rows	400 per parameter in the tree; the ticks for the rest still apply, and a parameter's checkbox still acts on all of them
ISO values named in a tooltip	8, then *+N more*
DATA auto-expand	200 runs. A larger file opens closed; every run is still there, and the choice you make is remembered
Sample cache	2048 MB by default, floored at 64 MB, trimmed to 80 % of the cap on overflow
Sidecar threshold	8 MB; below it, parsing is quick enough that a sidecar would save nothing measurable
Formula dependency depth	32, beyond which the definition is reported as cyclic
Redraw coalescing	25 ms, so a drag produces one resample at its end rather than one per pixel
Caption commit delay	400 ms of idle typing
PARAMS filter delay	400 ms of idle typing
Interface size	70 % to 160 % of the base type scale, in steps of 10
Shutdown wait	30 s for background work to finish

14.2 Getting the best out of it

Open each large file once and let the sidecar be written; the cost is paid on the first open and never again, and the status message says when it is happening. Prefer fewer, larger pages over many sparsely filled ones, since a fresh 5 × 5 page costs most of a second to build its widgets

while switching between pages of the same shape merely redraws. Where a tile shows many runs of the same channel, turn the legend's *File* and *Simulation* parts off so identical rows merge — sixty rows of legend cost more to lay out than the curves do. And if the cache figure in the status bar is climbing towards the cap on a machine with plenty of memory, raise the cap with `-memory-limit`: eviction is safe, but a column that is evicted and re-read is a column parsed twice.

14.3 Troubleshooting

14.3.1 The program will not start

Run `python -m visimu -selftest`. It checks each dependency separately and then the core, so the first `FAIL` names the problem. If the core is the failure, the error lists every directory that was searched; build it, or point `VISIMU_CORE` at the library. On Windows, a load failure that looks like a missing dependency suggests running `objdump -p` against the DLL to see what it wants. Whatever happens, the traceback is in `visimu.log`, whose location is reported by `-version`.

14.3.2 A file opens but shows nothing

Opening succeeds as long as the file has a usable `*PARAMETRES` block, so a file with headers and no `*TABLEAU` opens with zero runs — the DATA tab will show the file with nothing beneath it. If runs are present but no curve appears, check in order: that runs are ticked in DATA — on a file of more than 200 runs the node opens closed, so click it open to see them; that no ISO filter is holding them back, which the status bar states explicitly; and that the tile has both an x channel and at least one y channel, which the tile's own title says in words.

14.3.3 The curves look wrong on a phase plane

A plot whose x channel is not monotonic — Mach against altitude over a climb and descent, for instance — needs the unsorted path through the decimator. The application always requests detection, so this is handled; but a script calling `decimate()` on such data should be aware that the sorted fast path would bracket only the first crossing. There is a related caveat in `nearest_index`: its fast path is chosen by comparing the first and last samples, which is a heuristic rather than a sortedness test, so a phase-plane channel that happens to end above where it started can return a wrong index.

14.3.4 A value I expected is missing or reads as nan

An unreadable token becomes not-a-number rather than zero, by design, so a gap in the file is a gap in the curve. Division by zero and the out-of-domain cases of `sqrt`, `asin`, `acos` and the logarithms also yield NaN rather than an infinity, because an infinity would destroy the plot's y range. In INSPECT, NaN prints as a dash.

14.3.5 A channel disappeared after I removed a computed parameter

Removing a computed parameter removes everything defined in terms of it, and the status message names what went. Re-create the dependency first, then the dependants.

14.3.6 Colours changed when I opened another file

They should not: a colour is allocated on first use and kept for the session, keyed on an identity that survives renaming — including the renaming of the *file*, and including a reload. If they

did change, something called *Re-assign palette*, which deliberately forgets the assignments so that the next redraw allocates afresh.

14.3.7 I reopened a workspace and the plot is empty

Both halves of what gets drawn are restored, so start by reading the status bar. If it says runs are held back by the ISO filters, open ISO: a filter whose node reads *no run satisfies it* is doing exactly what it was asked to and the data no longer contains anything in that band, so widen or untick it. A filter reading *not in the loaded files* is filtering nothing and is not the cause, and one reading *paused* is not applying at all.

If the status bar says no runs are plotted at all, the DATA tab has nothing ticked — the workspace records which runs were chosen by index, so opening it against a file with fewer runs leaves the missing ones simply absent.

The dialog shown at load time, *Workspace loaded with differences*, lists everything the workspace asked for that this data could not give: unopenable files, formulas that could not be defined and why, aliases whose channel is gone, and both ISO cases. If it did not appear, everything in the workspace resolved.

14.3.8 I swapped one study for another and the axes moved

They should not, since 2.4.0 — see §6.4. If a tile's x axis did move, the status bar says so by name (*page 2, tile 3: x axis VTAS -> TIME*), and it means the new files genuinely do not carry the channel that tile was plotting against.

Appendix A

Keyboard shortcuts and gestures

The *Help* button in the toolbar shows this list in the program, beside a line on what each tab is for.

A.1 Keys

Keys	Action
Ctrl+N	New workspace — offers to save the open one first
Ctrl+L	Load workspace — offers to save the open one first
Ctrl+S	Save workspace (overwrites the current file)
Ctrl+Shift+S	Save workspace as — asks, then continues in the new file
Ctrl+0	Auto-fit every tile
Ctrl+- / Ctrl+=	Interface size: smaller, larger
Ctrl+1 … Ctrl+4	Mouse mode: Select, Inspect, Pan, Zoom
↑ / ↓	Previous or next tab in the rail
D	DATA
P	PARAMS
C	COMPUTE
F	DIFF
O	ISO
S	STYLE
A	SCALE
Y	SYNC
3	3D
I	INSPECT
R	AREA
M	MARKERS
T	TEXT

The tab letters are bare keys, with no modifier, and each is shown on its tab when the pointer is over the rail. A bare letter cannot be allowed to steal a keystroke meant for a field, so the keys are ignored whenever the focus is in a text box, a spin box, an editable combo or a text area, and whenever a modal dialog is up. The theme has no shortcut: *Light / Dark* in the toolbar is the only way, because a theme switch is not something to hit by accident.

A.2 Gestures

Gesture	Action
Click a tile	make it the active tile
Double-click a tile	activate it, fit it, and bring PARAMS forward
Double right-click on a tile	fit every tile on the page
Drag in Zoom	zoom the area, the x axis or the y axis, by the shape of the drag
Ctrl +drag or right-drag in Zoom	apply the same x range to every tile on the page
Wheel in Zoom	zoom about the pointer
Drag in Pan	move the view
Drag in Inspect	measure a slope (shorter than 6 px counts as a click)
Click a curve in Inspect	read every channel of that run at that x
Drag a marker or band edge in Select	move it
Drag the legend	place it freely; the corner becomes *custom*
Drag a caption in Select	move it within its tile
Drag a tab name up or down	rearrange the rail
Wheel over the tab names	previous or next tab
Double-click a row in DATA	rename that file or run
Double-click a name in PARAMS	plot it, or stop plotting it
Double-click a row in COMPUTE	load that definition for editing
Double-click a line in the comparison report	generate the pages for that run
Click a file row's arrow in DATA	open or close its runs; the choice is remembered
Right-click in DATA	plot all or no runs of a file; rename; reload from disk; copy the full path
Right-click in PARAMS	use as x axis; copy the name
Right-click in ISO	group, ungroup, select all or none, use this filter, remove the filter
Drag on a 3D tile in Pan	rotate
Middle-drag or Shift +drag on a 3D tile	translate
Right-drag or Ctrl +drag on a 3D tile	zoom
Wheel on a 3D tile in Zoom	zoom

Appendix B

Environment variables

Variable	Effect
`VISIMU_CORE`	Path to the core library, or to a directory containing it. Searched first, before every built-in location
`VISIMU_LOG_DIR`	Directory for `visimu.log`
`AOE_CACHE`	Set to `0`, `false`, `no` or `off` to disable sidecar caching entirely. Read once per process
`AOE_CACHE_DIR`	Overrides the per-user directory used for sidecars and for HDF5 layout files

Appendix C

The workspace file

A workspace is UTF-8 JSON with the extension `.aoews`, written with two-space indentation. The schema is at version 3.

```
{
  "version": 3,
  "app_version": "2.4.0",
  "files":    ["/abs/path/run_a.aoe", "/abs/path/run_b.aoe"],
  "computed": {"LOAD_FACTOR": "NZ * 9.80665"},
  "file_names": {"/abs/path/run_a.aoe": "Nominal climb"},
  "sim_names":  {"/abs/path/run_a.aoe":
                   {"run_a.aoe::SIM_001::0": "Cold day"}},
  "pages": [
    {"name": "Page 1", "rows": 2, "cols": 2, "tiles": [ ... ]}
  ],
  "active_page": 0,
  "selected_sims": [[0, 0], [0, 1]],
  "styles": {
    "colors":  {"param::ALPHA": "#e8232a"},
    "dashes":  {"file::/abs/path/run_a.aoe": "dash"},
    "symbols": {"param::ALPHA": "o"},
    "widths":  {"param::ALPHA": 2.0},
    "line_width": 1.6,
    "symbol_size": 6.0
  },
  "iso_states": [
    {"param": "INITIAL_MASS", "active": true, "grouped": true,
     "group_count": 3, "value_enabled": [],
     "groups": [["57236 - 64495", 57236.3, 64495.9, true],
                ["64495 - 71754", 64495.9, 71753.9, false]]}
  ],
  "markers": [
    {"x_param": "TIME", "position": 5.0, "visible": true,
     "color": "#ff3333", "dot_size": 9.0}
  ],
  "areas": [
    {"x_param": "TIME", "x_min": 10.0, "x_max": 20.0,
     "color": "#3380ff", "alpha": 0.2, "visible": true}
  ],
  "global_x_limits": {"TIME": [0.0, 60.0]},
  "global_y_limits": {},
  "dark_mode": true,
  "ui_scale": 1.0,
  "panel_order": ["DATA", "PARAMS", "DIFF", "STYLE"],
  "mouse_mode": 0,

  "diff_file_a": 0,
  "diff_file_b": 1,
  "diff_config_path": "/abs/path/thresholds.yaml",
  "diff_rules": [
```

```
    {"group": "BLACK", "param": "BETA", "x_param": "TIME",
     "threshold": 10.0},
    {"group": "AMBER", "param": "BETA", "x_param": "TIME",
     "threshold": 5.0}
  ],
  "diff_group_colors": {"BLACK": "#9aa3ad", "AMBER": "#e59b3a"},
  "diff_report": {
    "file_a": 0, "file_b": 1,
    "name_a": "regression_a.aoe", "name_b": "regression_b.aoe",
    "flags": [
      {"run_id": "RUN_001", "param": "BETA", "x_param": "TIME",
       "violations": [
         {"group": "BLACK", "threshold": 10.0,
          "regions": [[15.0, 20.0]]},
         {"group": "AMBER", "threshold": 5.0,
          "regions": [[10.0, 15.0], [20.0, 25.0]]}
       ]}
    ],
    "warnings": []
  }
}
```

Each entry of `"tiles"` carries the complete tile state listed in Chapter 6, including its captions and both encodings. Only the visible tiles of a page are written; loading pads the list back up to twenty-five.

`ui_scale` is the interface size as a multiple of the base type scale. A value outside the supported range is clamped rather than refused, and one absent altogether opens at 1.0.

`panel_order` is the arrangement of the tab rail, as panel names. A name this version does not have is ignored, and one it has that the list does not mention keeps its place at the end — so the rail is arranged as far as the file says and nothing is ever lost from it.

The `diff_*` keys are the file comparison (Chapter 5). `diff_file_a` and `diff_file_b` index `"files"`. `diff_rules` is the configuration *as read*, which is why the path alone is not enough: the file on disk is maintained by whoever owns the thresholds, and a saved report has to go on meaning what it meant when it was computed. `diff_report` holds the flags, each violation carrying its group, its threshold and the x regions where that group had priority — already reduced, so nothing is recomputed on load. Each is checked on the way in and a malformed one costs the DIFF tab its contents rather than costing the workspace its pages.

A tile generated by the comparison carries `fdiff_run` (the run *identifier*), `fdiff_param`, `fdiff_file_a`, `fdiff_file_b` and `fdiff_bands`, a list of `[group, low, high]`.

`file_names` and `sim_names` are optional and written only when something was renamed (§7.3). `app_version` records which build wrote the file and is informational only.

`value_enabled` is a list of pairs rather than an object because JSON object keys are strings: round-tripping `0.75` through an object hands back `"0.75"`, which would silently stop matching the value it is about. Non-finite keys are dropped, so the file stays portable JSON.

The loader sets only the attributes it recognises and ignores the rest, which is what makes a hand-edited workspace, or one written by another build, open as far as it can be understood. The `"version"` key is written but nothing branches on it.

Not saved: the active tile index, the tile clipboard, the memory limit, the window geometry, and the hidden tiles of a shrunken grid.

Appendix D

Defaults at a glance

Setting	Where	Default
Window size	startup	1500 × 920
Theme	rail foot	dark
Interface size	toolbar	90 % (range 70–160 %, steps of 10)
Mouse mode	toolbar	Select
Grid	page bar	1 × 1
Legend, Grid	STYLE	on
Log Y, Points	STYLE	off
Colour by	STYLE	**Parameter**
Style by	STYLE	None
First three curve colours	STYLE	red, blue, dark green
Line width	STYLE	1.6 (range 0.1–10.0)
Marker size	STYLE	6.0 (range 1.0–20.0)
Legend corner	STYLE	top-left
Legend parts	STYLE	Parameter only
Axis locks	SCALE	off
Group colours	DIFF	grey, red, orange, green, in the order the groups appear
Comparison x axis	threshold file	`TIME` when an entry does not name one
Generated page grid	DIFF	4 × 4
Align runs on an event	SYNC	off
ISO groups	ISO	3
Azimuth, tilt	3D	45°, 25°
3D triad, 3D grid	3D	on
3D zoom half-width	3D	0.95
Band opacity	AREA	0.20 (range 0.02–1.0)
Band colour	AREA	`#3380ff`
Marker dot size	MARKERS	9.0 (range 2.0–30.0)
Marker colour	MARKERS	`#ff3333`
Caption size	TEXT	10.0 pt (range 4–72)
Caption position	TEXT	0.05, 0.05
Memory limit	command line	2048 MB (floor 64 MB)

Appendix E

Known limitations

The following are real shortcomings of this version, gathered here so they are easy to find.

An HDF5 file's layout is inferred, and reading one needs h5py. The tree is evidence rather than proof: a file following none of the known conventions opens with its runs and channels arranged wrongly, or does not open at all. The log names the convention chosen and what else was considered, which is where to look first; see section 8.7.

A pushed file cannot have a sidecar written until every column of it is resident. The core writes one from what it holds and has no reader of its own to fill a gap, so the separate process that writes the sidecar for an HDF5 file fetches the whole file first. On a file larger than the machine's memory that fails, and no sidecar is written — the file is still usable, column by column.

The workspace version is not interpreted. A `"version"` of 3 is written and nothing reads it; there is no version-branching logic, so a future format change has no mechanism to fall back on. What stands in for it is the loader's tolerance of unknown keys and of missing ones.

Group boundaries differ by a hair between the two layers. The core treats every interval except the last as half-open; the interface helpers treat all of them as inclusive at both ends with the first claiming group winning. The two disagree only for a value sitting exactly on a shared edge.

`nearest_index` guesses at sortedness. Its fast path is chosen by comparing the first and last samples rather than by testing monotonicity, so a phase-plane channel that ends above where it started can return a wrong index.

`aoe_cache_path` writes shared state under a read lock. Two concurrent readers can invalidate each other's returned pointer. The Python layer is not exposed to it because `ctypes` copies the string immediately.

There is no cancellation. A running background task cannot be interrupted; the interface is disabled until it completes. What exists instead is coalescing — a newer queued request replaces an older one.

Markers and areas are not drawn on three-dimensional tiles. This is a deliberate restriction rather than an oversight, but it is a restriction.

Two threshold groups with the same number are tied. Priority in the file comparison is the threshold value, so two groups sharing one have no order between them and which gets shaded is undefined. The program warns rather than choosing silently — when the configuration is read, and again in the report — but it cannot resolve it; see §5.4.1.

Classic Mac line endings are not supported. Lines are split on the line feed, with carriage returns trimmed, so a file using lone carriage returns reads as a single line.

The DATA tree holds one widget item per run. A twelve-hundred-run file is twelve hundred of them. The cost is paid once per change to the set of open files rather than per interaction, and not auto-expanding a large file keeps Qt from laying them out — but the items are built either way, and a model-backed view is what would avoid that.

Decimation scans the whole visible window. That is what makes the min/max guarantee hold, but it means a fully zoomed-out view of a large study scans every sample of every drawn curve on each resample. A multi-resolution min/max pyramid stored in the sidecar would make that case cost what the screen can show instead; it would also change the sidecar format.